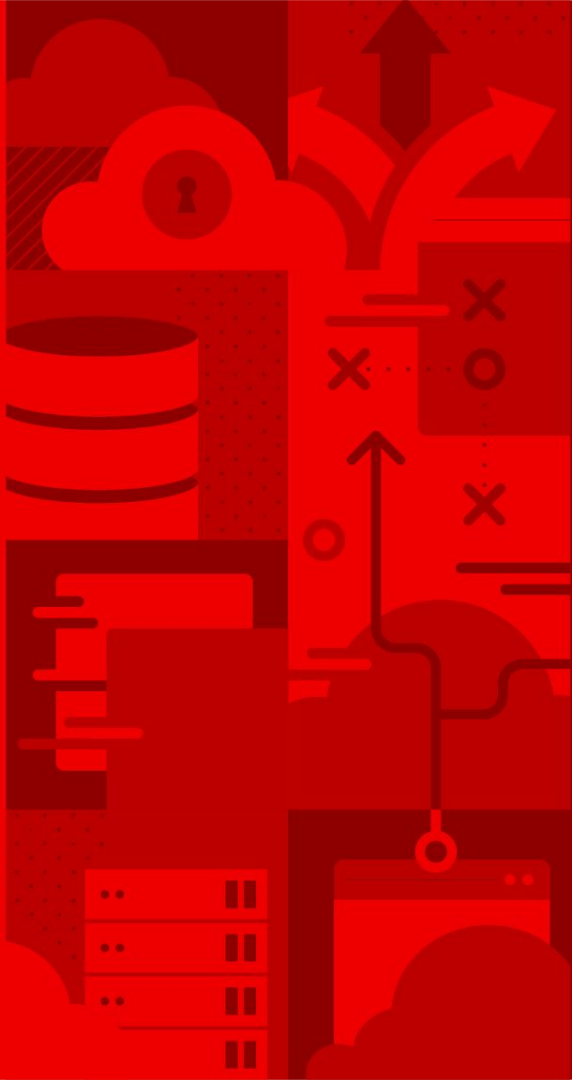
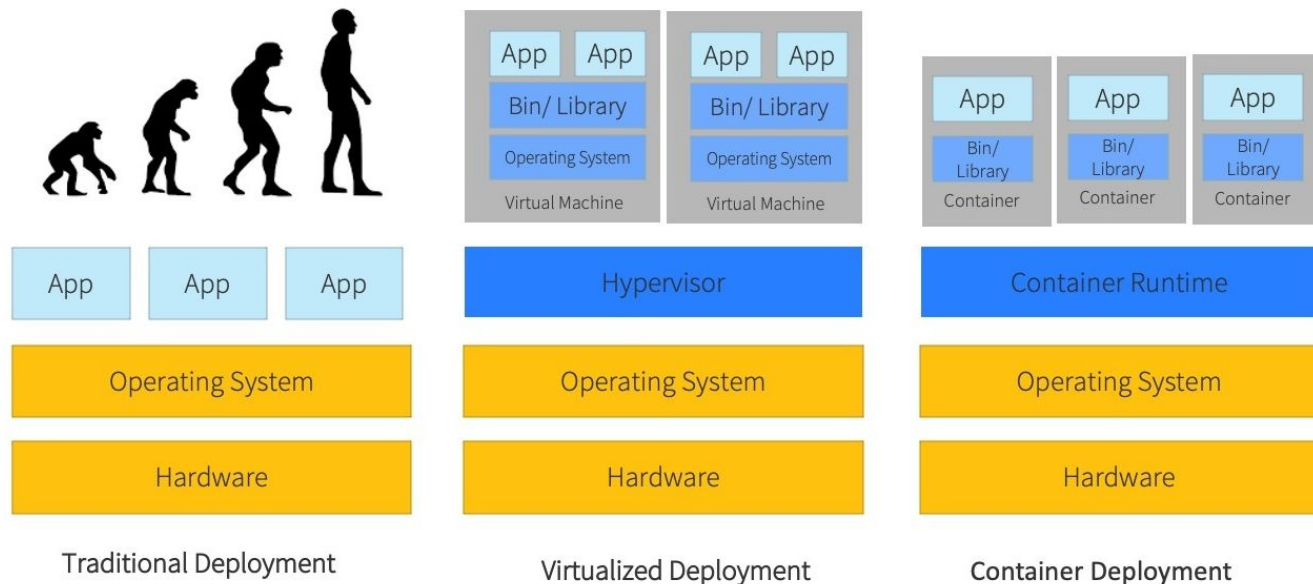


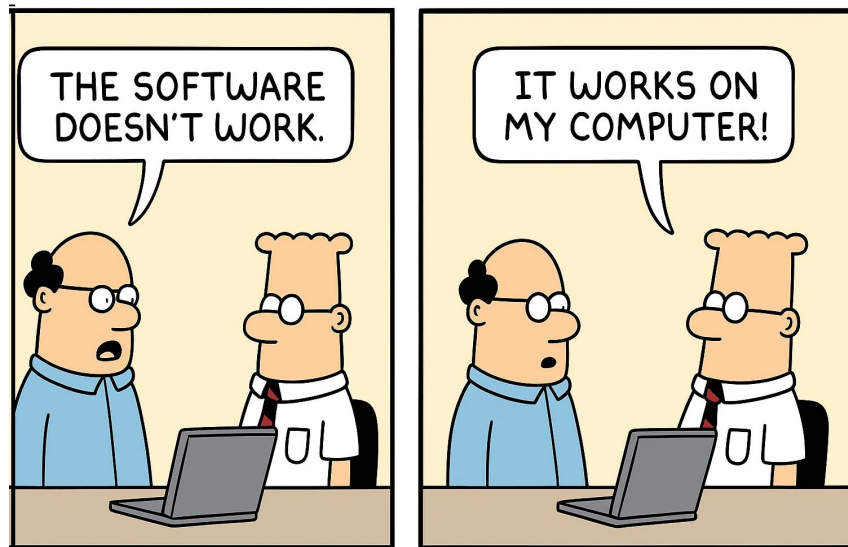


Containers Overview

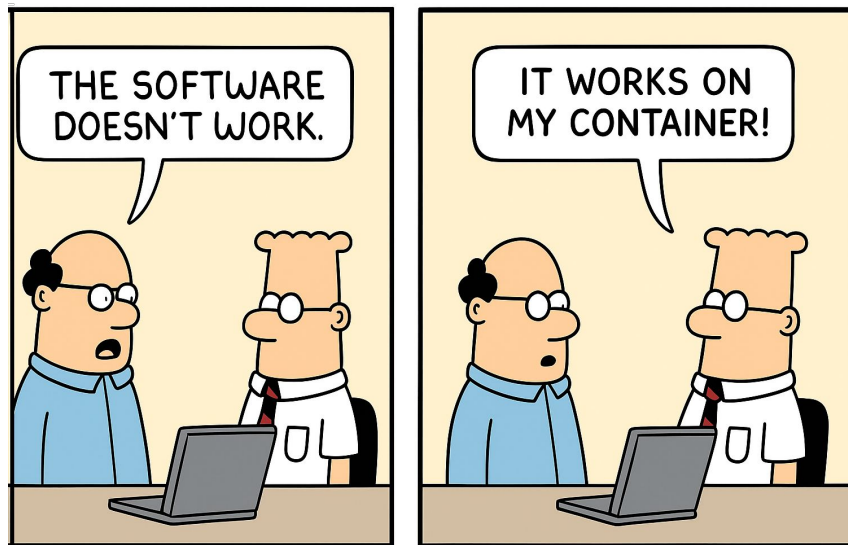
Evolution of Application Deployments



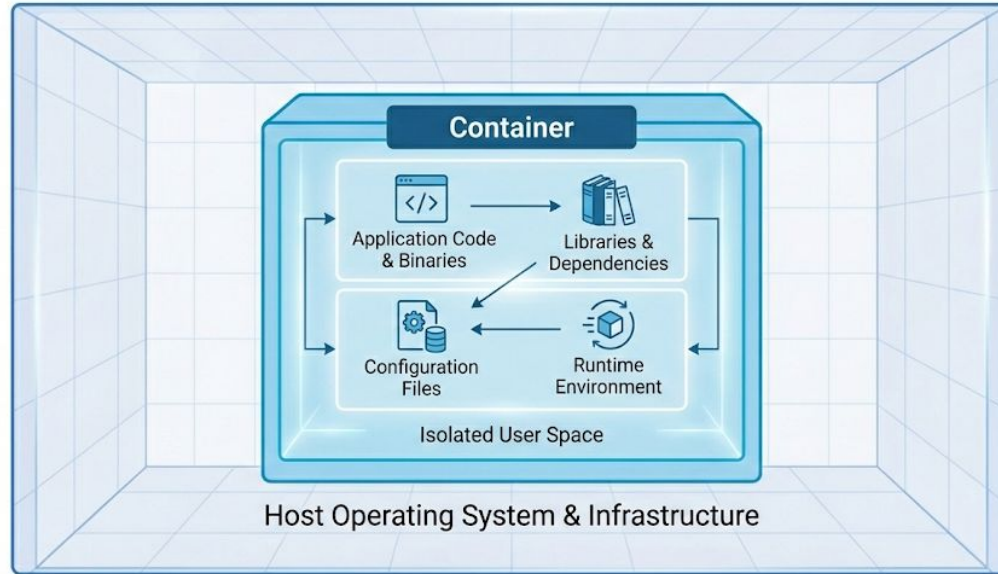
Prima della DevOps.....



.....dopo la DevOps



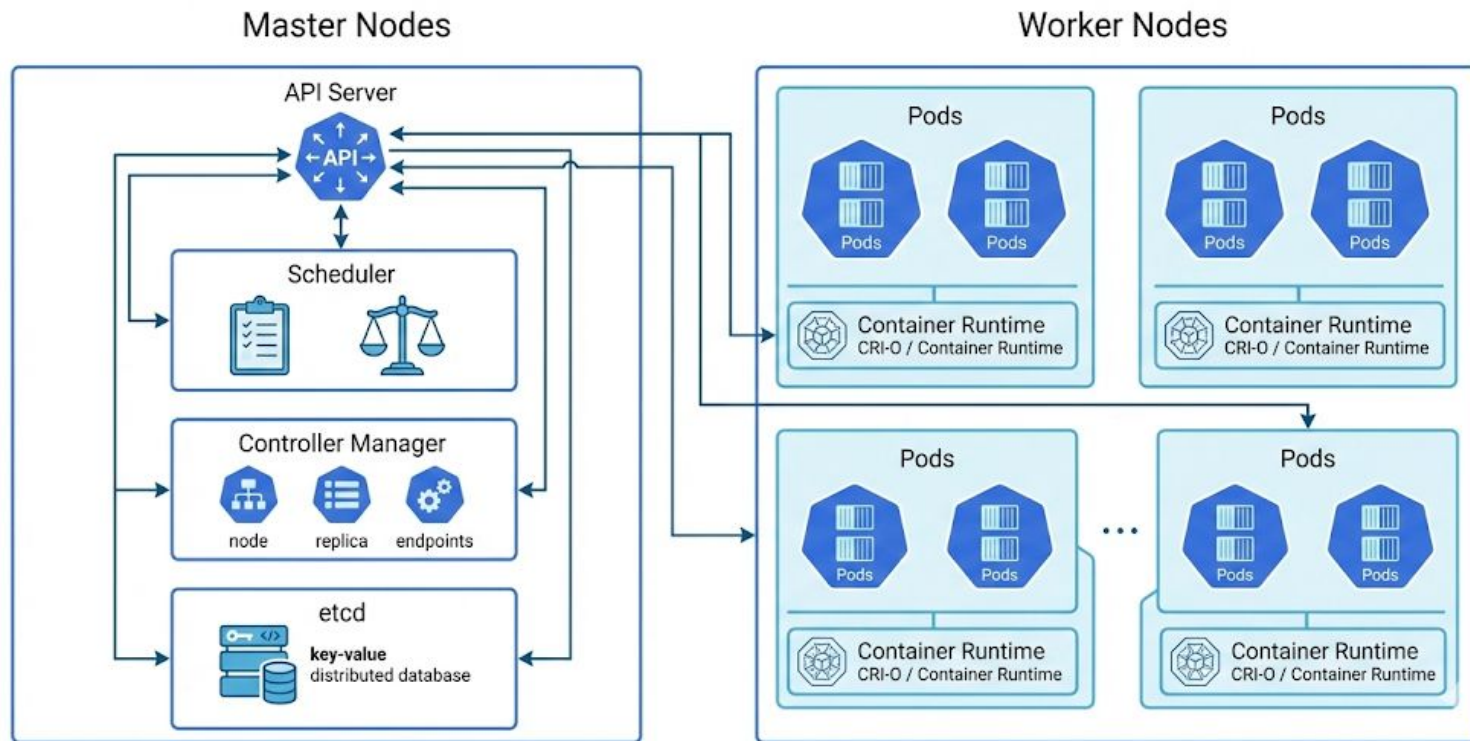
Cosa è un Container ?



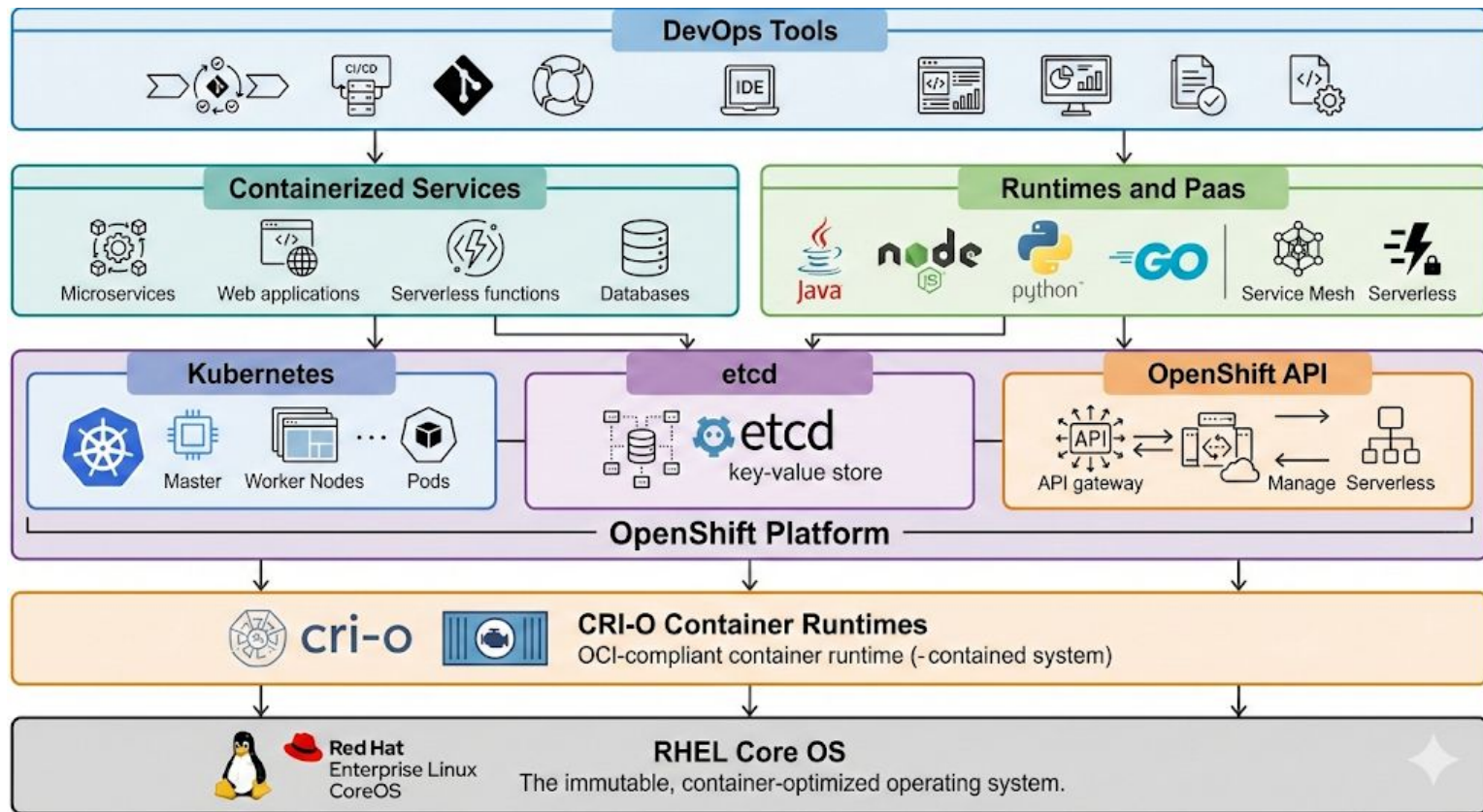
<https://youtu.be/7l3e5Fu-t6c>

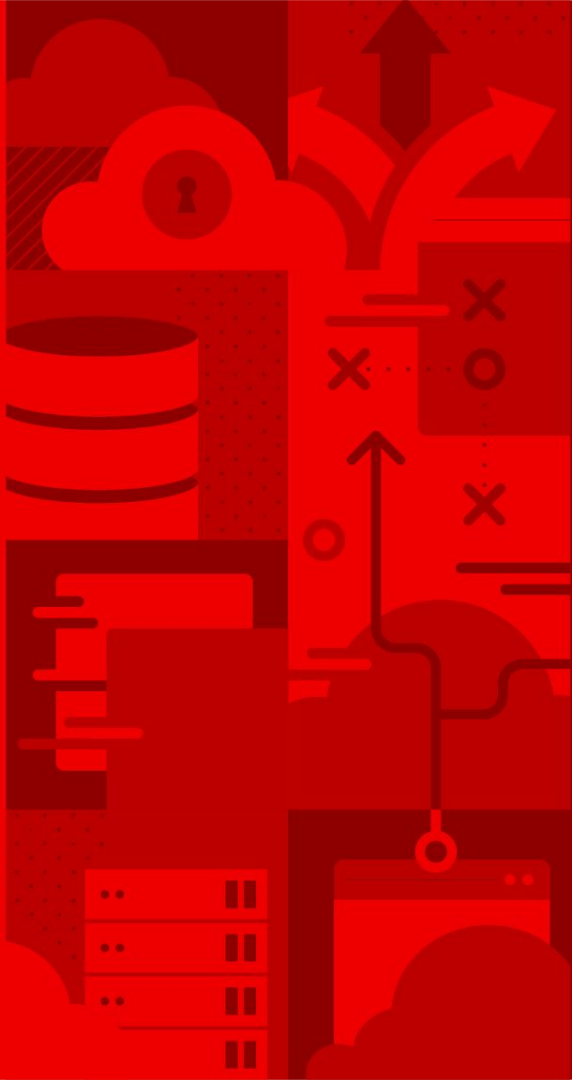
Containers Demo!

Kubernetes: Architettura



OpenShift: Architettura





Run Applications as Containers and Pods

Il Container è l'elemento più piccolo nel mondo Kubernetes



CONTAINER

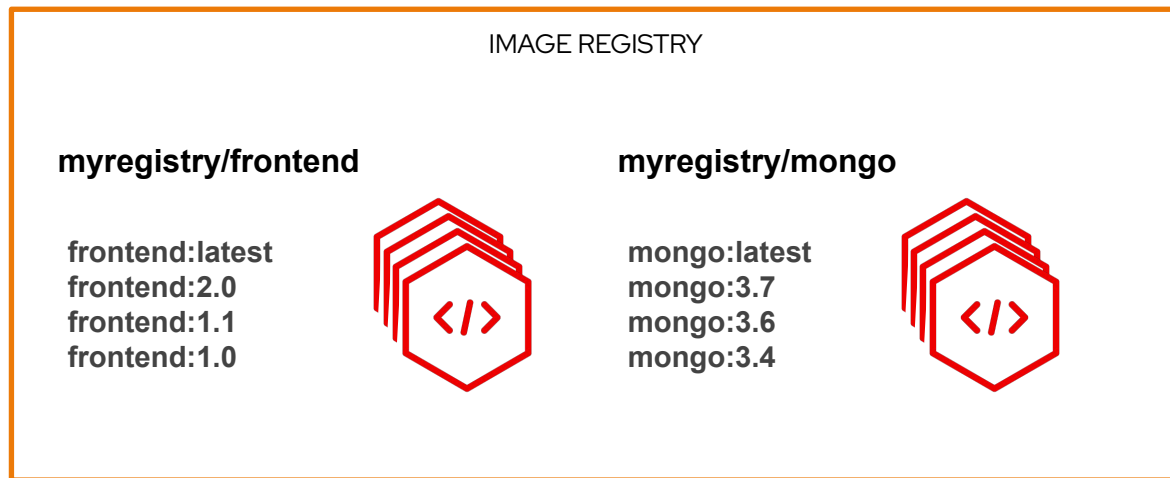
I Containers sono creati dalle Immagini



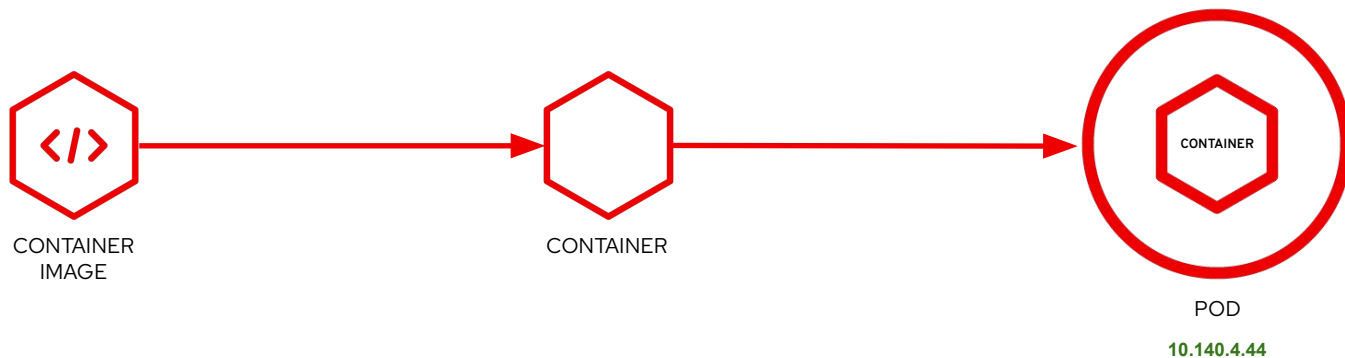
Le Immagini sono conservata in un Registry



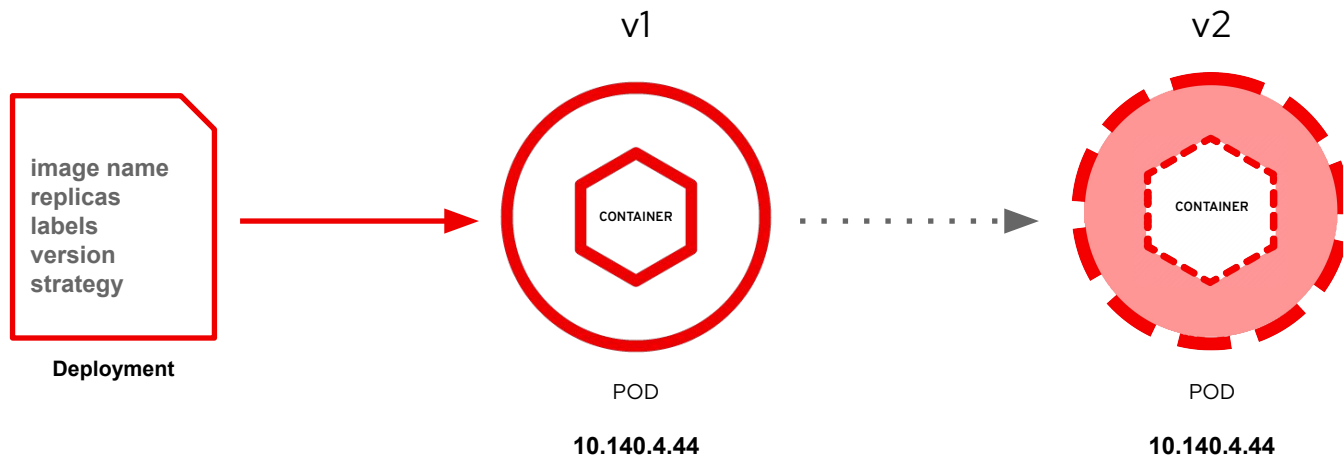
Il Registry contiene tutte le versioni delle Immagini



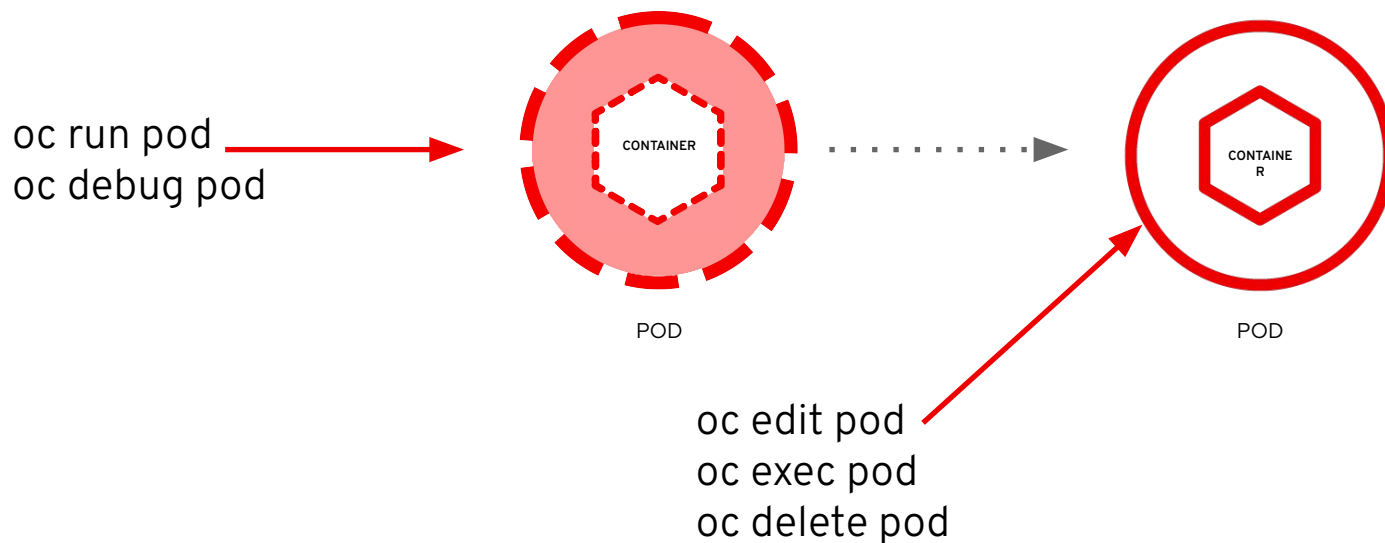
In OpenShift i workloads girano nei Pod



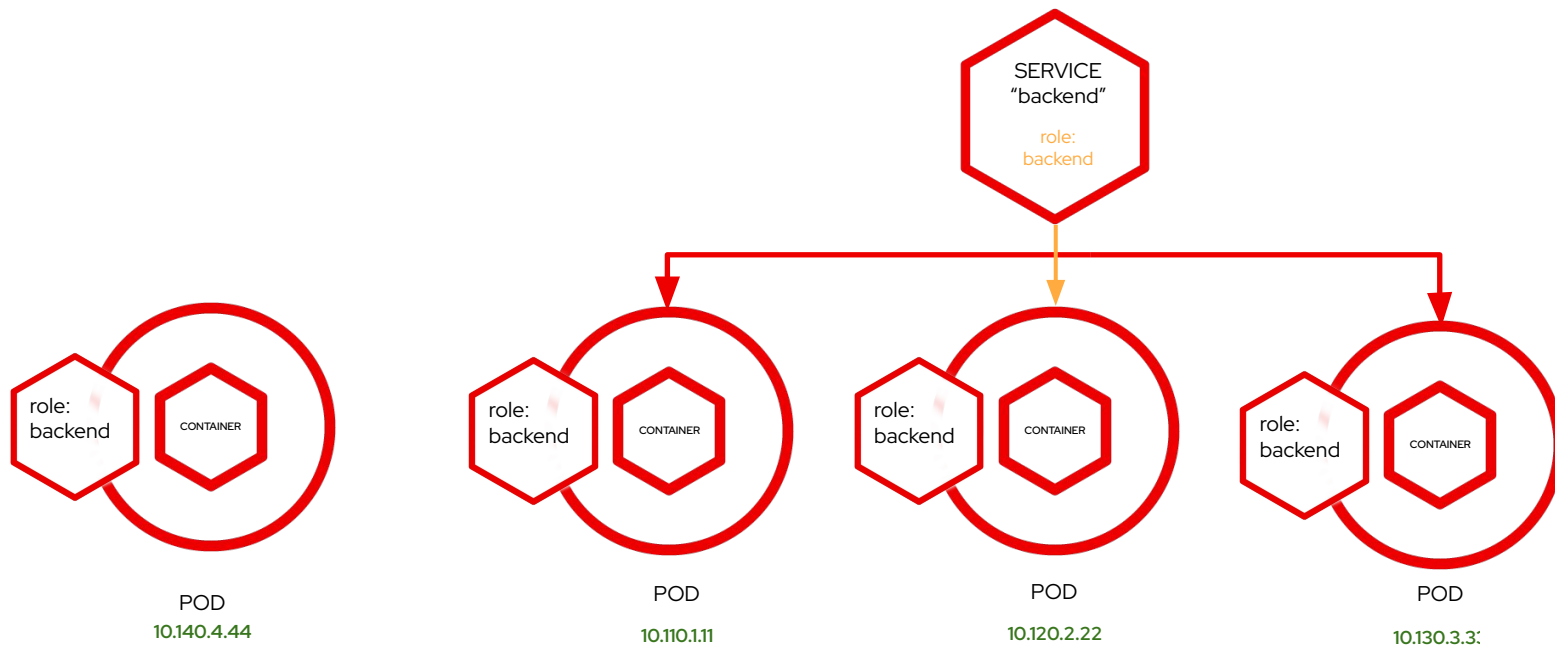
I Pods possono essere governati dai Deployments



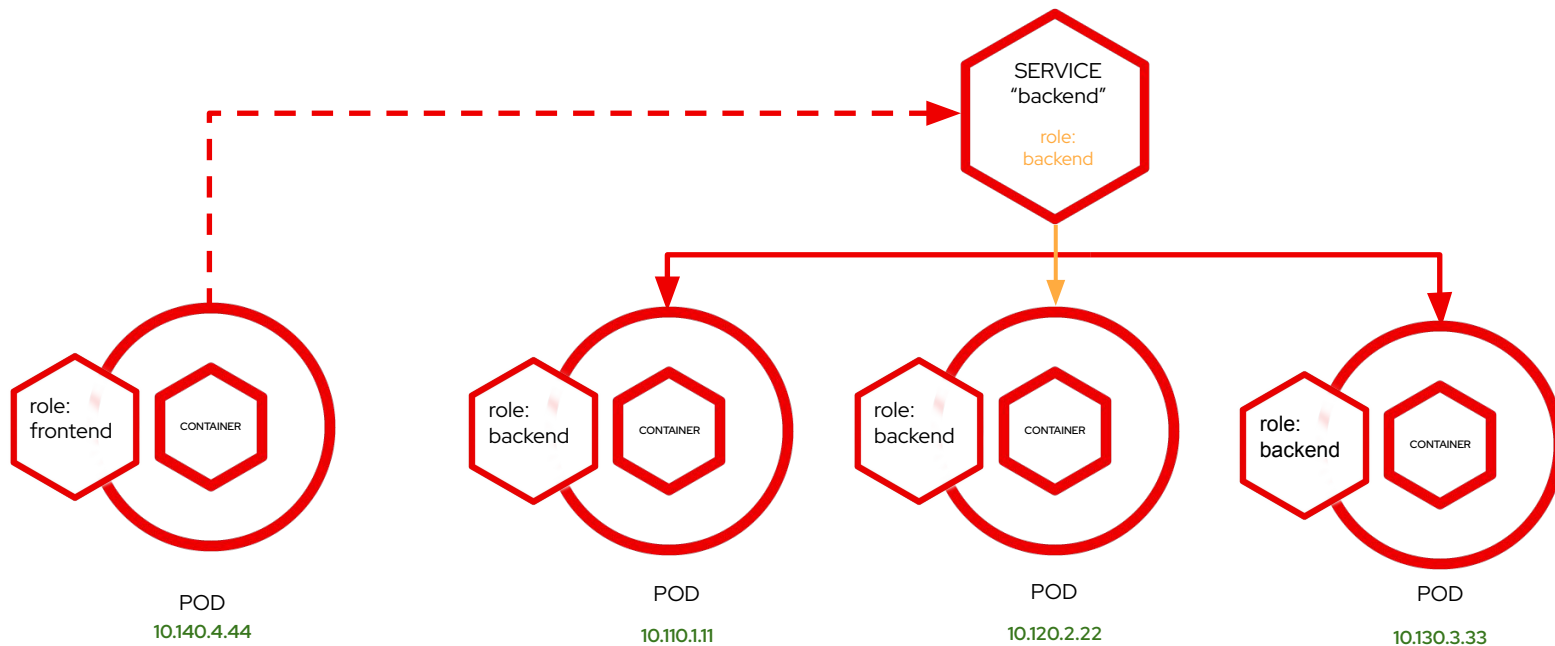
Comandi di gestione dei Pods



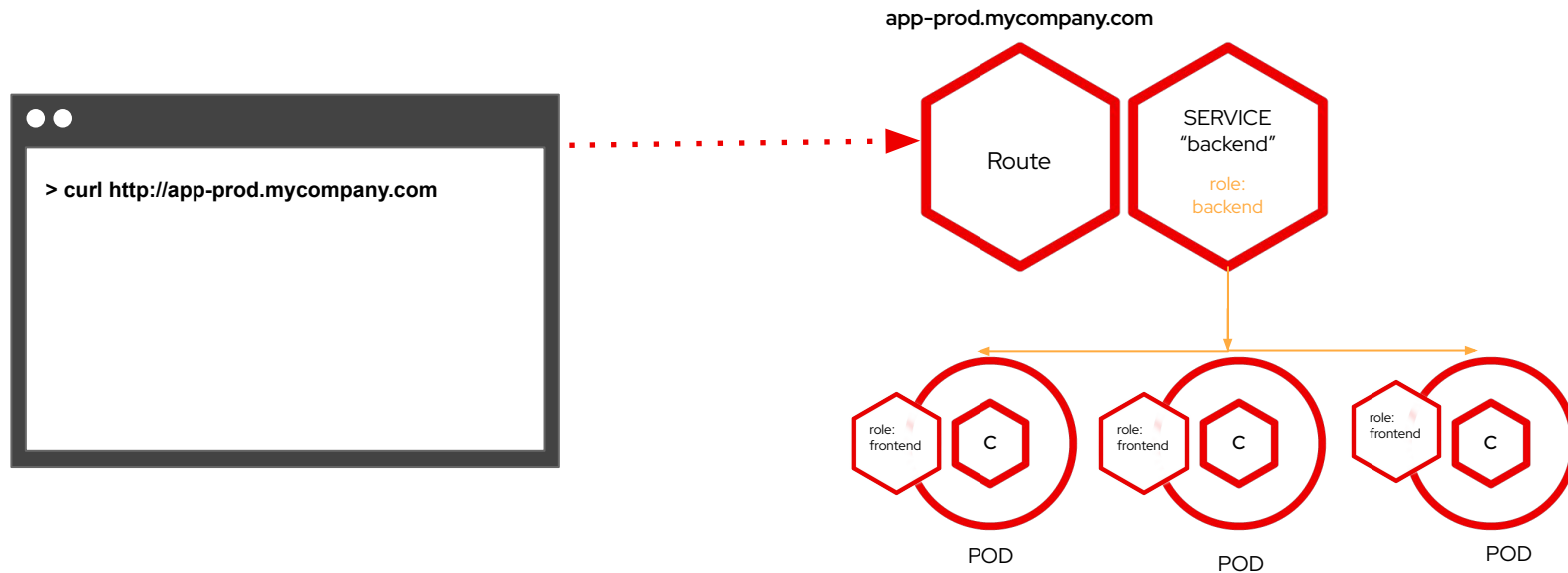
I Services distribuiscono le chiamate tra Pods



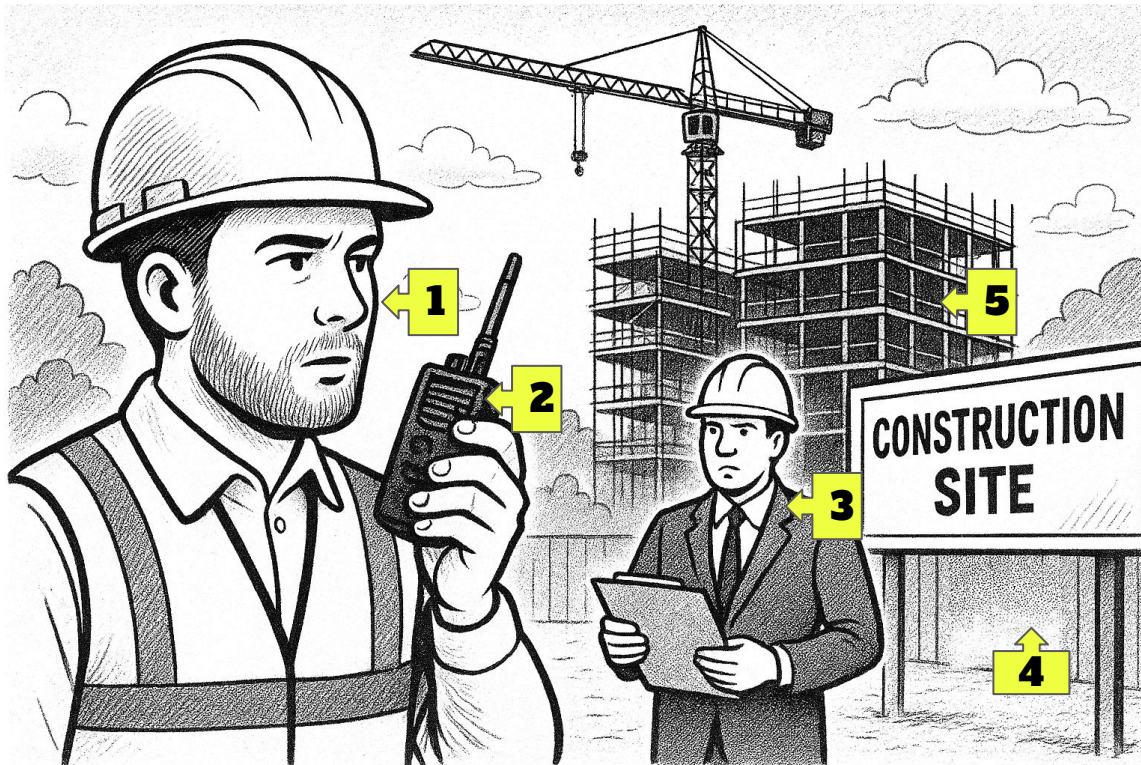
Le applicazioni comunicano mediante Services



Le Route rendono i Service accessibili ai client esterni tramite HTTP/S



Quiz: Se OpenShift fosse un cantiere ?



SERVICE
"backend"

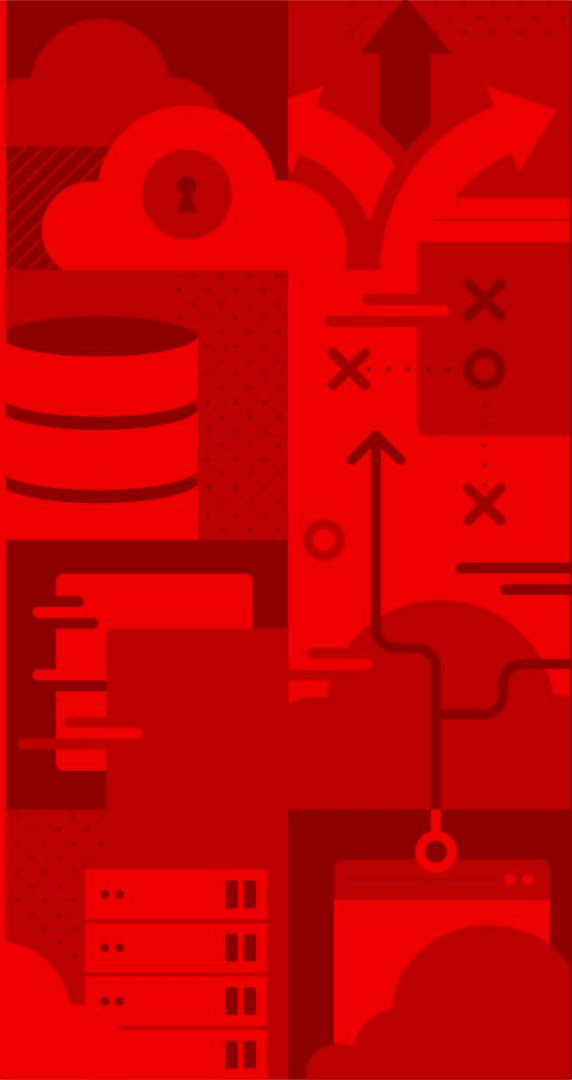
image
name
replicas
labels
version
strategy

Deployment

C

POD

Route
app-prod.acme.com



Deploy Applications on Kubernetes

Openshift Offering



Self-Managed Editions

- Red Hat OpenShift Container Platform
- Red Hat OpenShift Kubernetes Engine
- Red Hat OpenShift Virtualization Engine

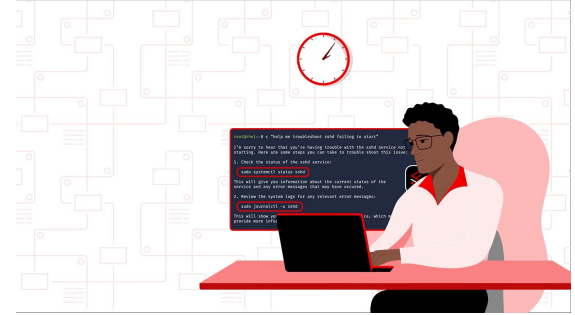
Managed Editions



Gestione risorse su OpenShift

Gestione Imperativa

- Comandi dalla CLI per creare, modificare e cancellare le risorse
- Esecuzione con il tool **oc** (openShift client): **oc expose service httpd**



Gestione Dichiarativa

- Utilizzo dei Manifest YAML o JSON per definire lo stato desiderato
- Uso tipico: **oc apply -f <file>.yaml**



Come è fatta una risorsa Kubernetes ?

```
apiVersion: v1
kind: Pod
metadata:
  name: my-sample-pod
  labels:
    app: my-app
    env: dev
spec:
  containers:
    - name: my-container
      image: nginx:latest
      ports:
        - containerPort: 80
      resources:
        requests:
          memory: "64Mi"
          cpu: "250m"
```

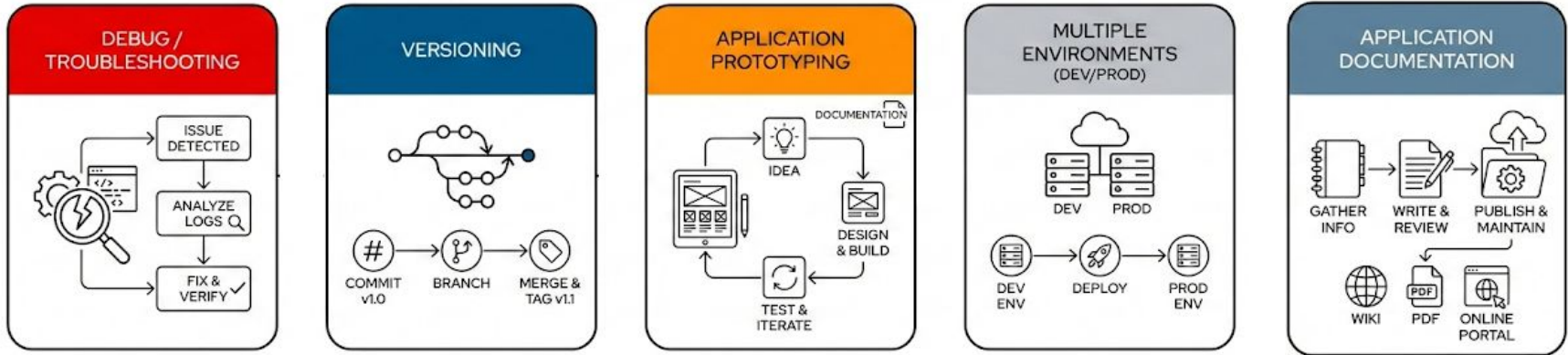
- **apiVersion:** identifica la versione dello schema dell'oggetto.
- **Kind:** indica il tipo di risorsa.
- **labels:** coppie chiave-valore per categorizzare e selezionare le risorse.
- **annotations:** metadati non identificativi (es. informazioni di versione, link).
- **spec:** definisce la configurazione richiesta per la risorsa.

Casi d'uso

Gestione Imperativa

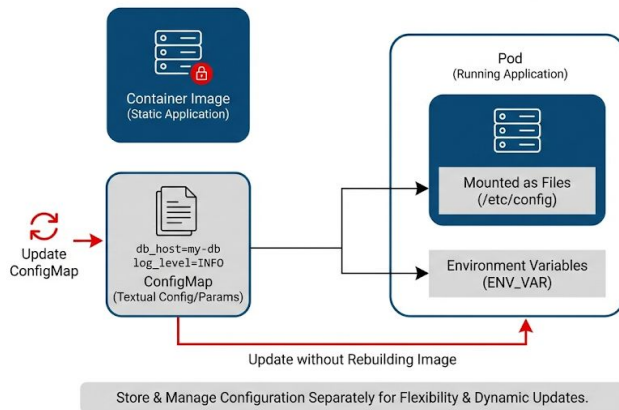
oc

Gestione Dichiarativa (YAML)



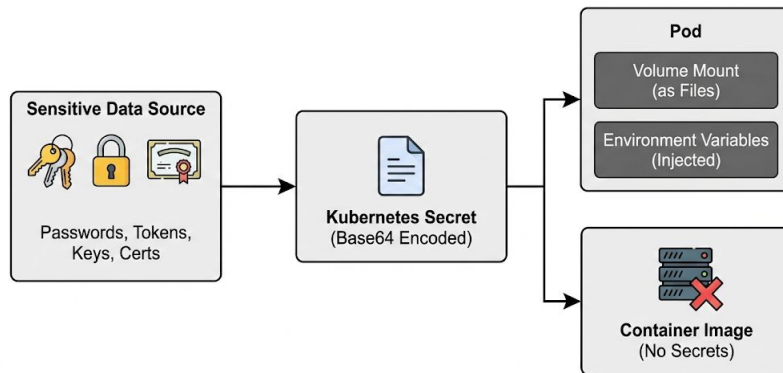
ConfigMaps

- Configurazioni testuali o parametri applicativi separati dall'immagine del container.
- Possono essere montate come file o esposte come variabili d'ambiente nei Pod.
- Permettono di aggiornare la configurazione senza ricostruire l'immagine dell'applicazione.



Secrets

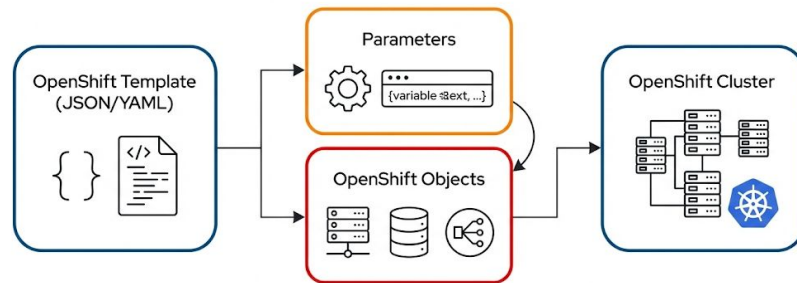
- Conservano informazioni sensibili come password, token, certificati e chiavi, **separandole** dall'immagine del container.
- Possono essere montati come file o esposti come variabili d'ambiente nei Pod, con accesso controllato.
- Sono codificati in base64 e conservati in etcd.



OpenShift Templates

I Templates sono modelli JSON/YAML riutilizzabili che definiscono oggetti OpenShift con parametri.

-  Integrati nativamente in OpenShift
-  Semplici da usare e condividere
-  Possono definire più risorse correlate
-  Usano parametri per personalizzare i workload



OpenShift Templates: Comandi

Controllo dei Templates disponibili:

```
oc get templates -n openshift
```

Creazione di un Template:

```
oc create -f <template-file.yaml> -n <namespace>
```

Processing di un Template:

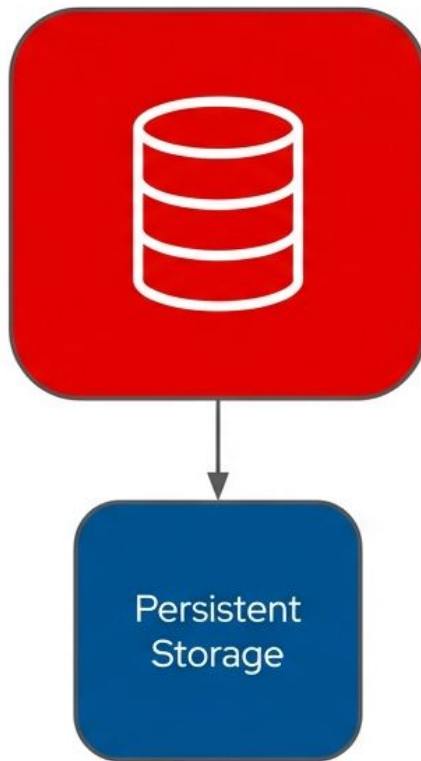
```
oc process <template-name> -p PARAM=value
```

Creazione di una Applicazione da Template:

```
oc new-app --template=<template-name> -p PARAM=value
```

Stateful su OpenShift: Dal Guscio Vuoto all'Architettura di Produzione

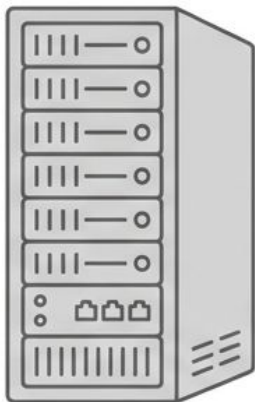
Gestione, Configurazione e Persistenza per Database Cloud-Native



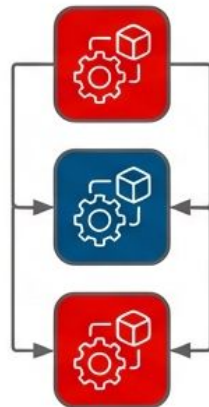
L'esecuzione di carichi di lavoro stateful (es. MySQL) su OpenShift richiede un radicale cambio di paradigma rispetto alle macchine virtuali tradizionali.

Il Cambio di Paradigma: Machine Virtuali vs. Container

Macchine Virtuali Tradizionali



Architettura OpenShift



Installazione manuale tramite procedure classiche e prolungate.

Deployment

Istanziamento immediato da immagini pre-pacchettizzate scaricate da un container registry (es. registry.ocp4.example.com). Garantisce portabilità e coerenza.

Script interattivi eseguiti post-installazione.

Configurazione Iniziale

I container nascono vuoti e sono immutabili. Richiedono configurazione automatizzata all'avvio.

Storage accoppiato al sistema operativo.

Gestione dello Stato

Il file system interno è temporaneo (effimero). Richiede disaccoppiamento esplicito dei dati.

La Sfida dell'Immutabilità: Il Guscio Vuoto

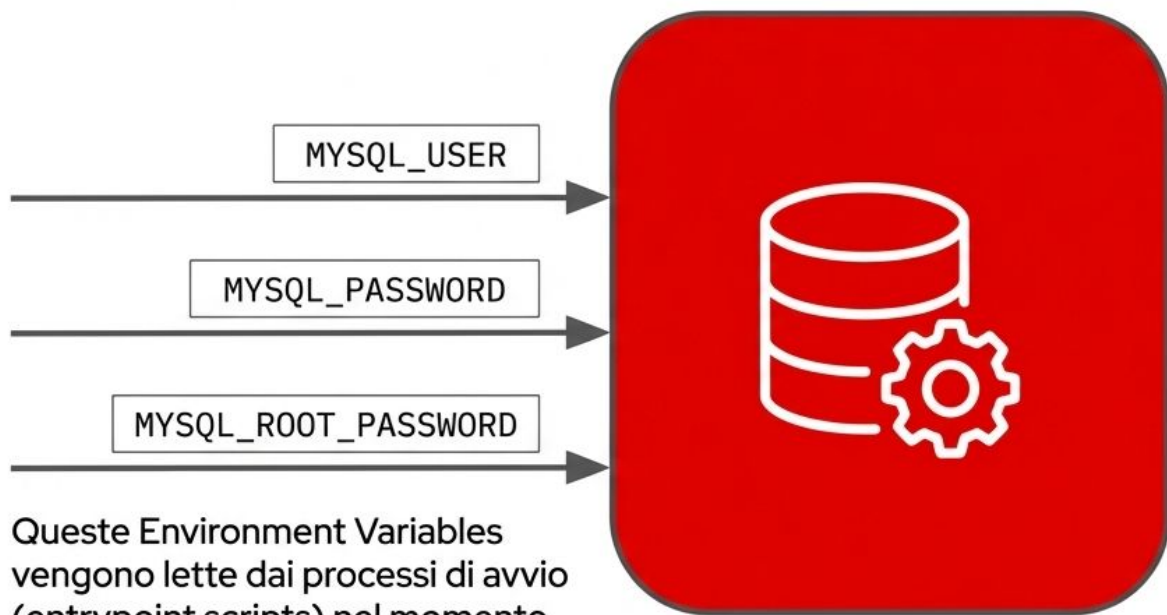
L'immagine MySQL scaricata non possiede un database di default, né credenziali di root o utenti applicativi.



I container cloud-native sono immutabili per design. È impossibile accedere manualmente al loro interno per lanciare script di configurazione interattivi durante il primo avvio.

Come si trasforma un guscio generico in un database funzionale senza intervento umano?

Iniezione della Configurazione (Environment Variables)



Queste Environment Variables vengono lette dai processi di avvio (entrypoint scripts) nel momento esatto della creazione.

La **soluzione standard**: iniettare parametri specifici all'interno della definizione del Pod durante la creazione del Deployment.

Il database si auto-configura, imposta i permessi ed è pronto ad accettare connessioni in modo totalmente automatizzato e sicuro.

Il Limite Critico: L'Illusione dello Storage Effimero

Stato Operativo



Il database elabora query e salva record nello storage effimero del Pod. Tutto sembra funzionare.

L'Evento di Riavvio



Il Pod viene cancellato, riavviato per un errore, o spostato su un altro nodo (scale-down/aggiornamento).

Perdita Totale

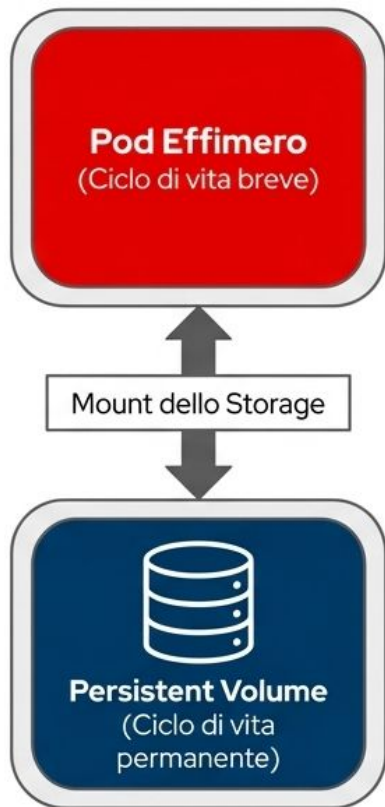


Il file system interno viene irrimediabilmente distrutto. Il database torna allo stato di fabbrica, perdendo ogni transazione.

Disaccoppiare il Ciclo di Vita: Persistent Volumes (PV)

In produzione, un database richiede obbligatoriamente l'aggancio a un Persistent Volume (PV).

- Il PV è l'unico strumento in grado di separare i dati dal Pod che li elabora.
- Se il Pod muore, i dati nel Deep Blue PV rimangono intatti e pronti ad essere montati dal Pod successivo.



Automazione e Manutenzione: L'Ecosistema Kubernetes

Il database è in esecuzione e accessibile tramite un Service (es. porta TCP 3306).

Invece di operare manualmente, OpenShift offre due potenti risorse.

Job (Attività Una Tantum)



Un task progettato per essere eseguito una sola volta fino al suo completamento con successo. Si avvia, esegue, e libera le risorse del cluster.

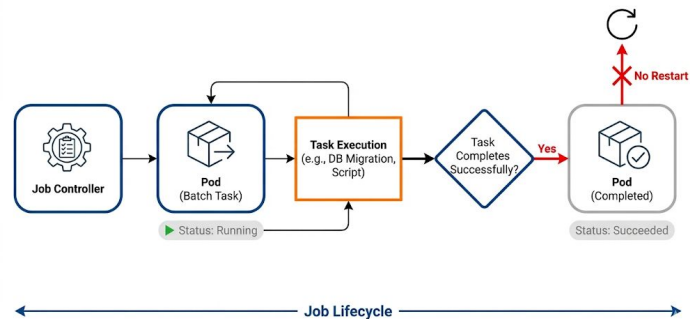
CronJob (Attività Ricorrenti)



L'equivalente cloud-native del crontab di Linux. Permette di schedulare l'esecuzione di Job a intervalli temporali regolari e ripetuti.

Esempio Job

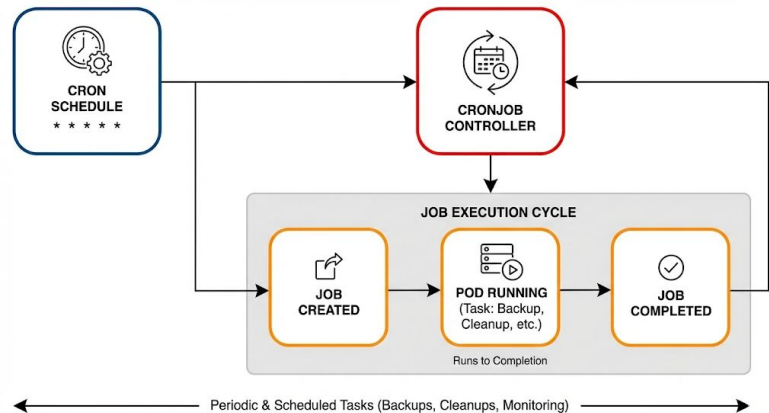
```
apiVersion: batch/v1
kind: Job
metadata:
  name: simple-job
spec:
  template:
    spec:
      containers:
        - name: simple-task
          image: busybox
          command: ["sh", "-c", "echo Hello
OpenShift; sleep 5"]
          restartPolicy: OnFailure
```



Esempio CronJob

- I CronJob creano Jobs in base a una pianificazione (come *cron* in Linux).
- La schedule usa la sintassi cron (** * * * **).
- Utili per task periodici, backup, pulizie o script di monitoraggio.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: example-cronjob
spec:
  schedule: "0 */6 * * *"      # every 6 hours
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: my-cron-job
              image: busybox
              command: ["sh", "-c", "echo Running
CronJob; sleep 10"]
              restartPolicy: OnFailure
```



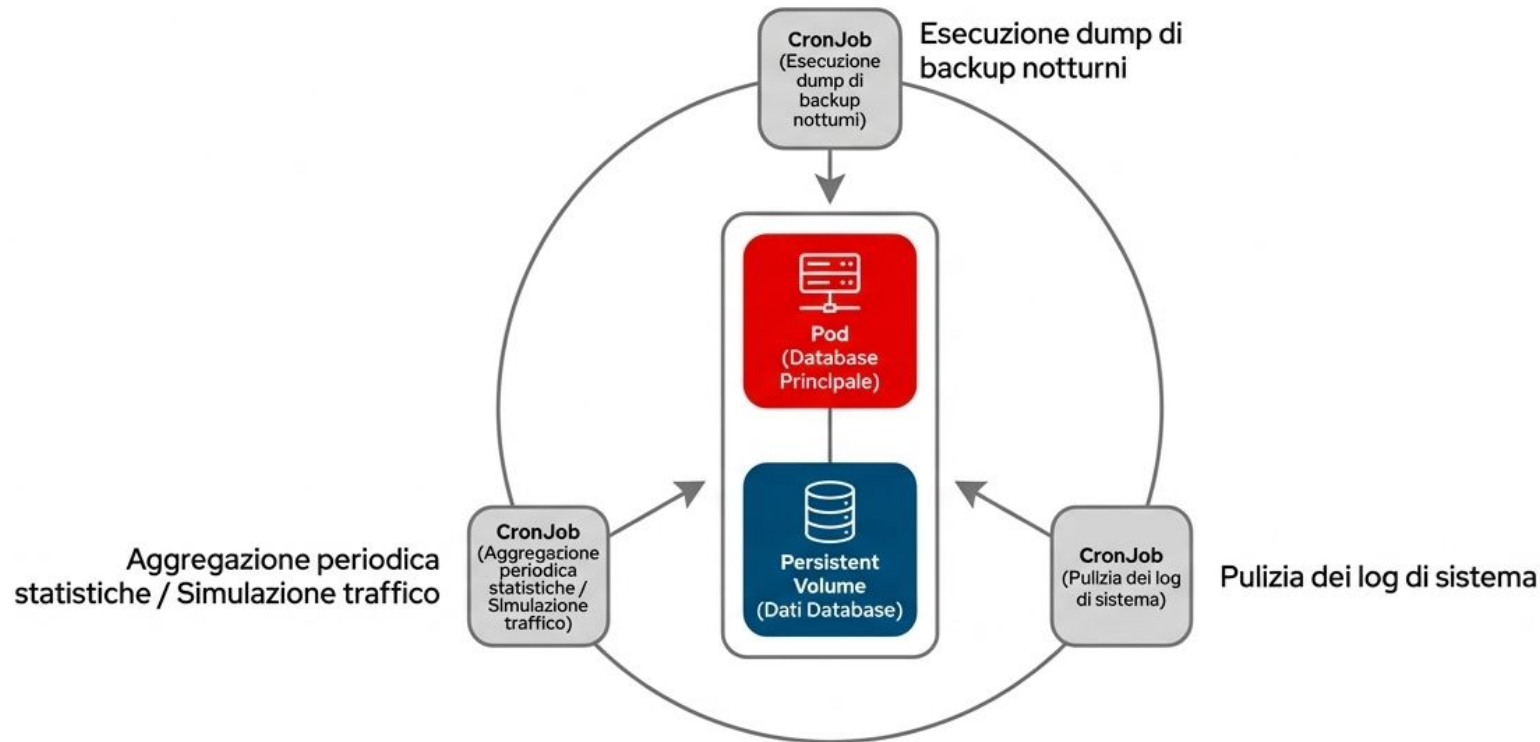
Job in Azione: Bootstrap e Inizializzazione



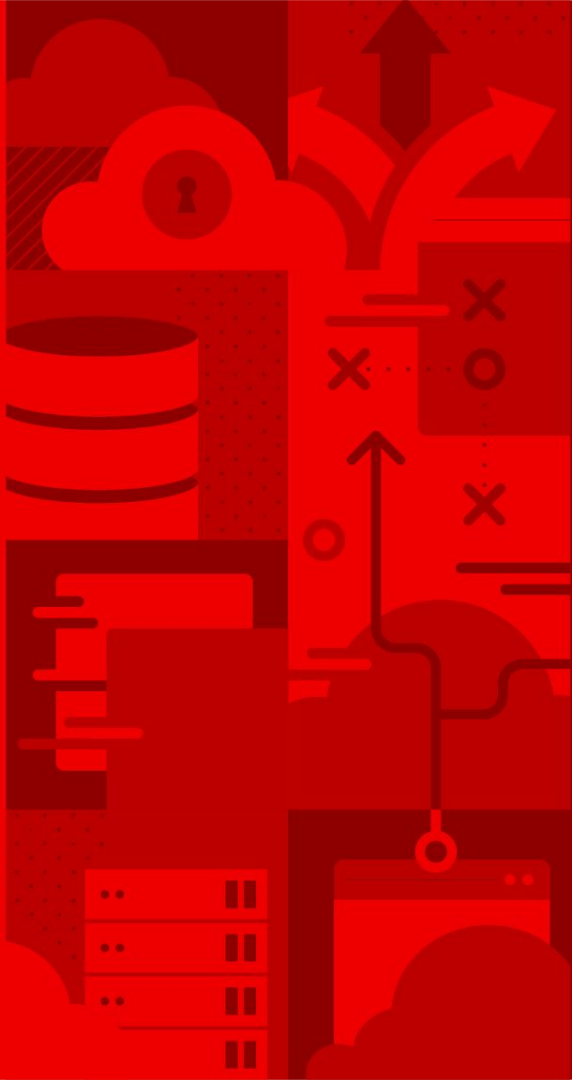
Scenario d'uso ideale per un Job: Operazioni di bootstrap.

Il Job lancia un client temporaneo che si connette al **database principale, struttura lo schema iniziale, e termina immediatamente**, liberando preziosa capacità di calcolo nel cluster.

CronJob in Azione: Il Motore di Manutenzione

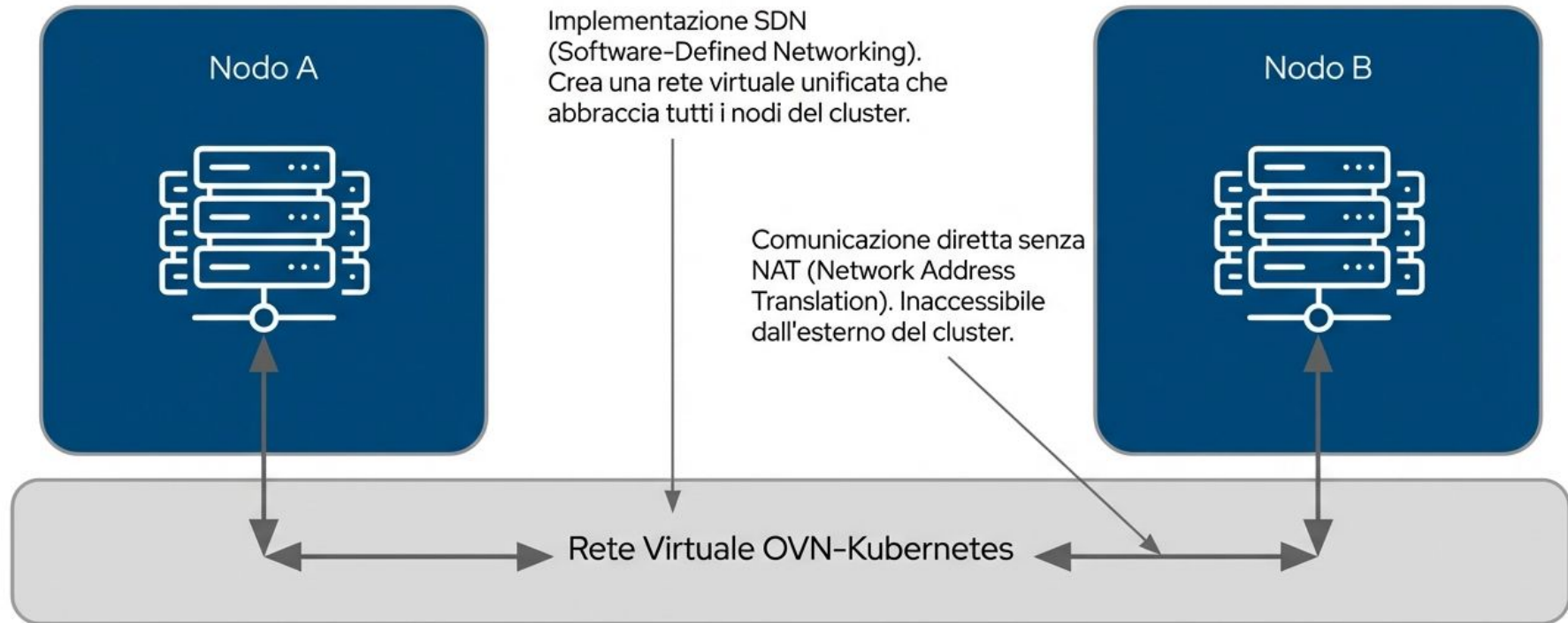


In scenari reali, il CronJob è il meccanismo d'elezione per automatizzare compiti cruciali di manutenzione senza intervento umano continuo.

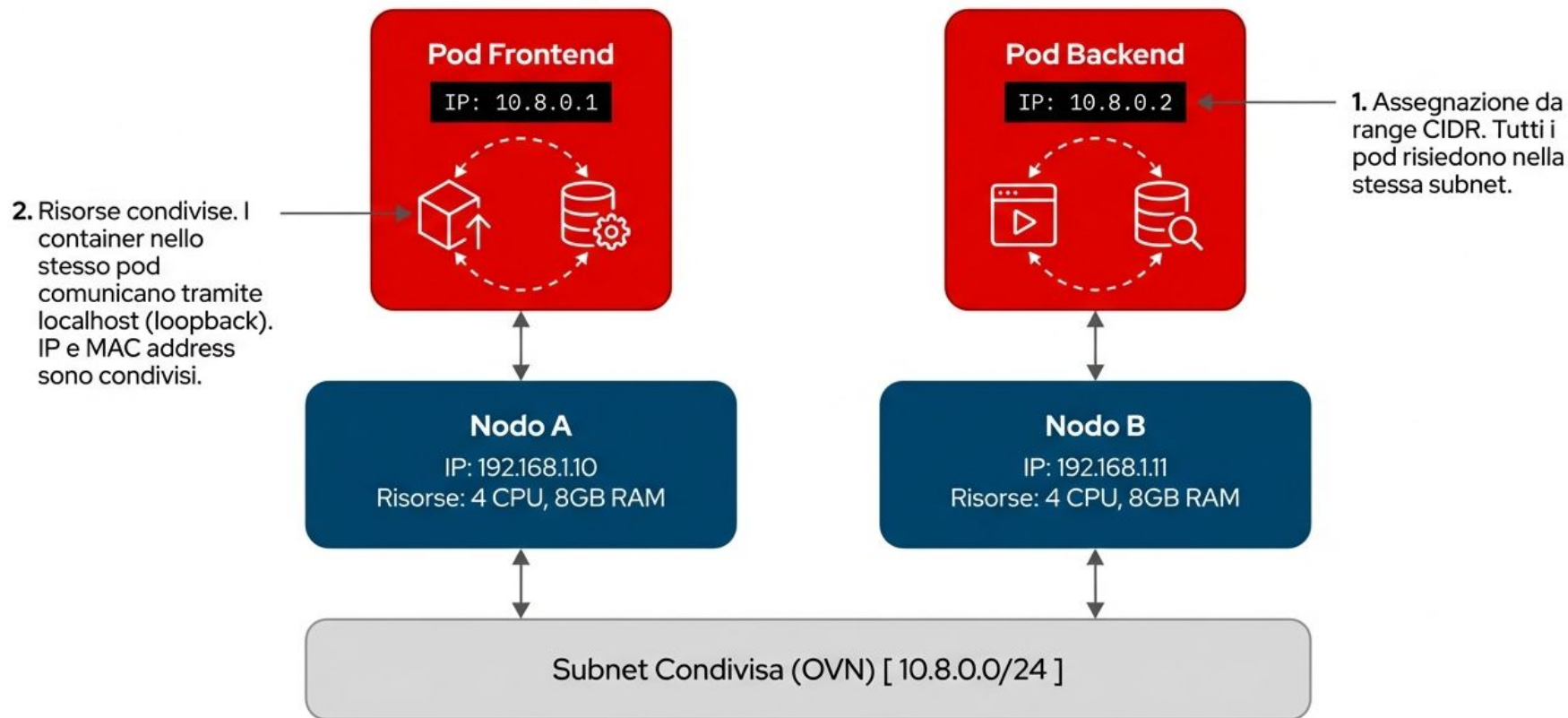


OpenShift Network

Architettura di Rete: Livello Base OVN-Kubernetes

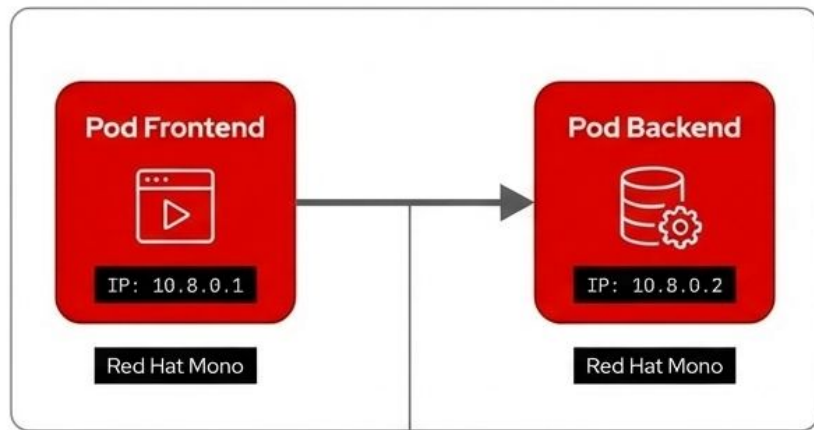


L'Ecosistema dei Pod: Subnet Condivisa



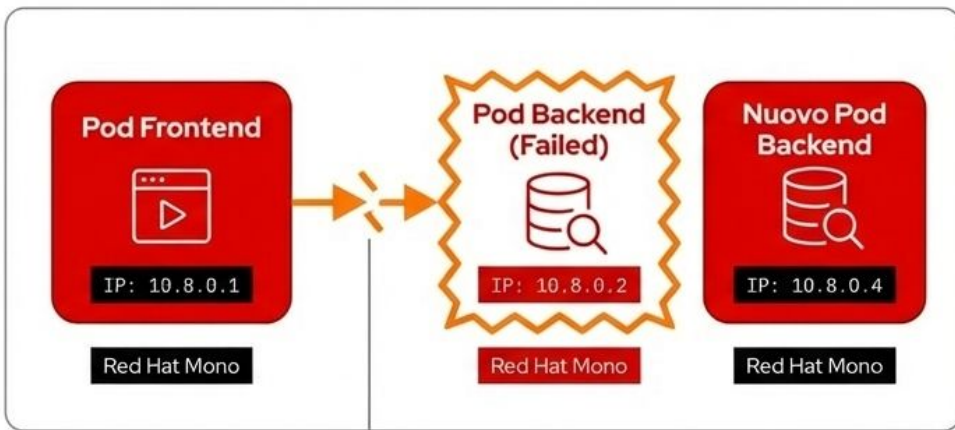
Il Dilemma della Volatilità: Fallimento del Pod

Prima



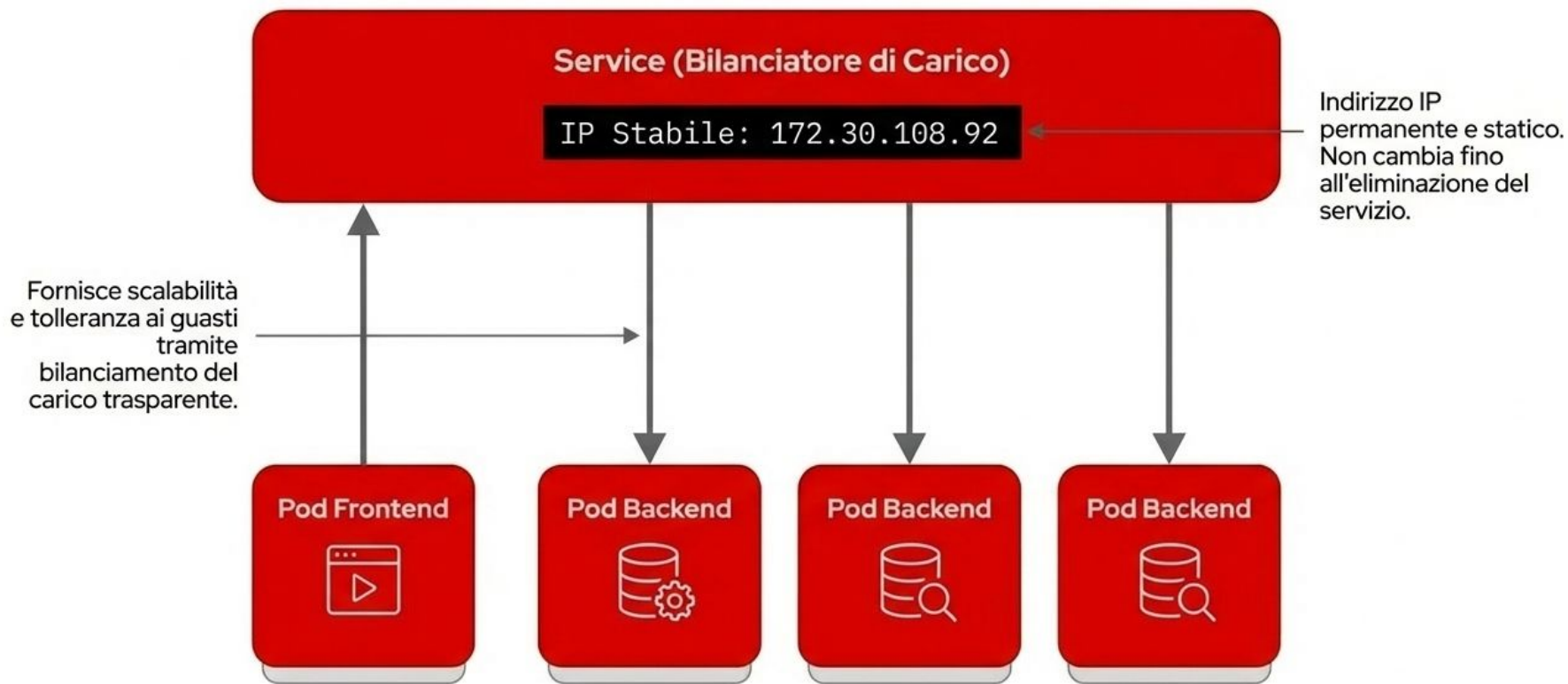
Connessione IP-to-IP
diretta.

Dopo

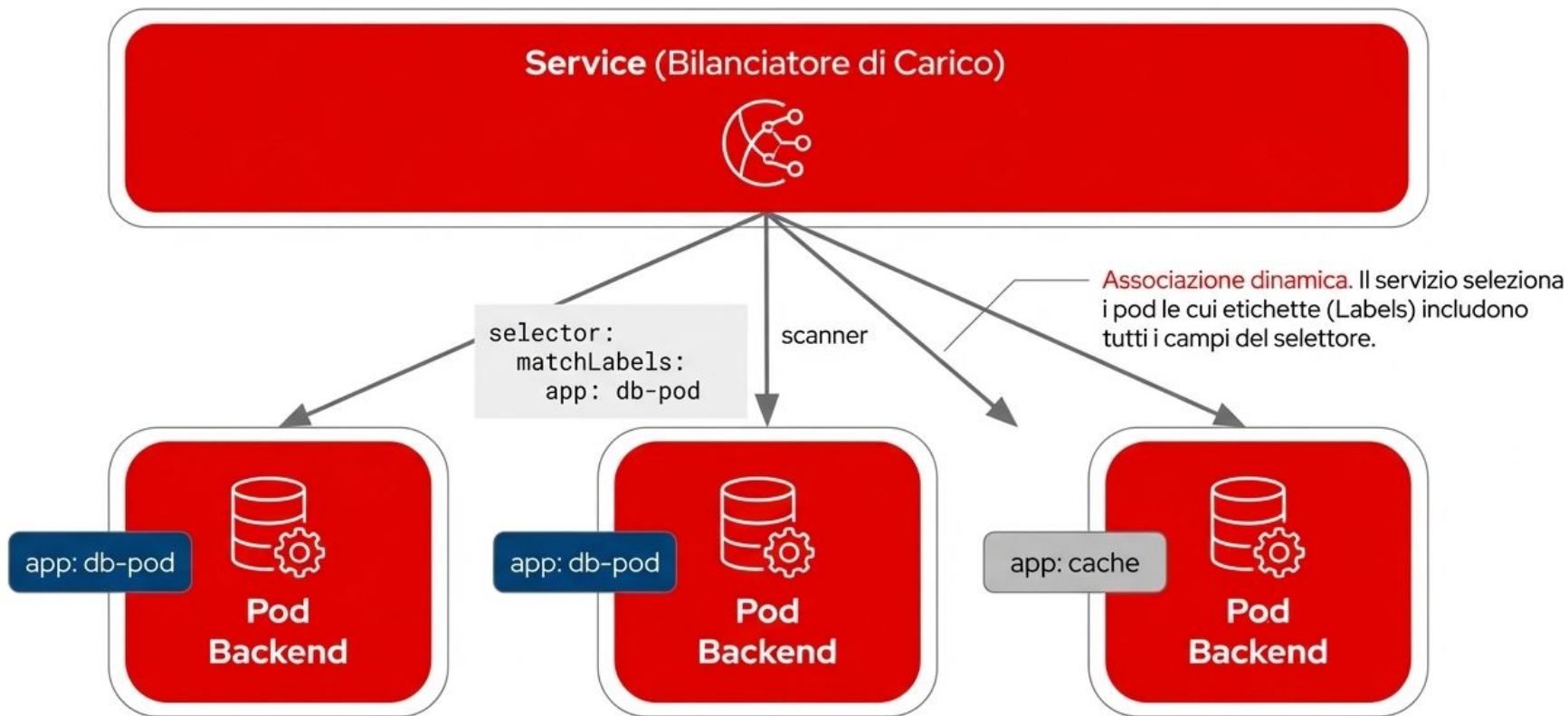


I pod sono effimeri.
I guasti o gli aggiornamenti
generano nuovi pod con IP
dinamici. Il riferimento del
Frontend diventa non valido.

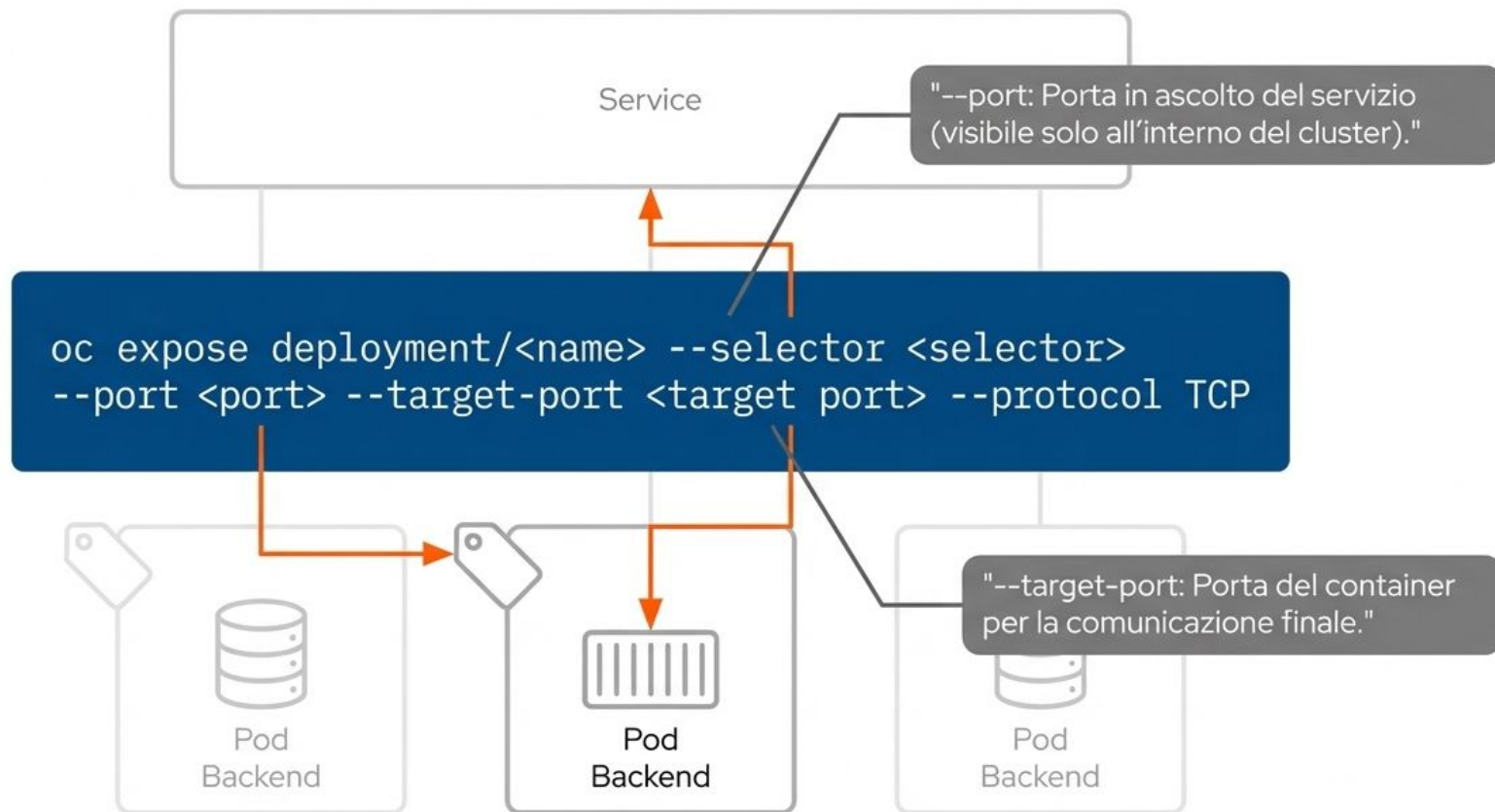
[L'Astrazione del Servizio: Load Balancer Virtuale]



[Selettori ed Etichette: Il Meccanismo di Routing]



[Esposizione del Servizio: Sintassi di Deployment]



[Verifica degli Endpoint: Ispezione del Traffico]



```
oc get service db-pod -o wide  
CLUSTER-IP: 172.30.108.92 | PORT: 3306/TCP |  
SELECTOR: app=db-pod
```



```
oc get endpoints db-pod  
ENDPOINTS: 10.8.0.86:3306, 10.8.0.88:3306
```

L'infrastruttura OVN-Kubernetes aggiorna dinamicamente l'elenco degli endpoint in caso di riavvio, replica o riprogrammazione dei pod.

[Service Discovery: Operatore Kube DNS (CoreDNS)]

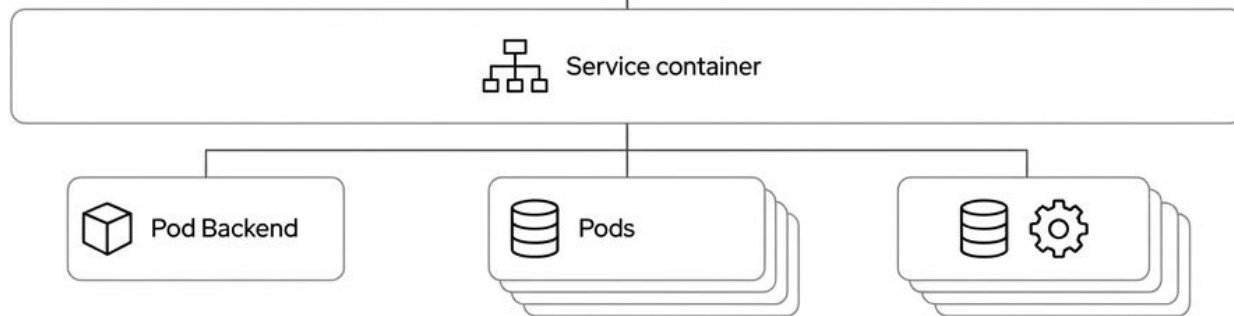
1

Distribuito dall'operatore DNS (operator.openshift.io). Assegna nomi DNS dinamici ai servizi definiti.

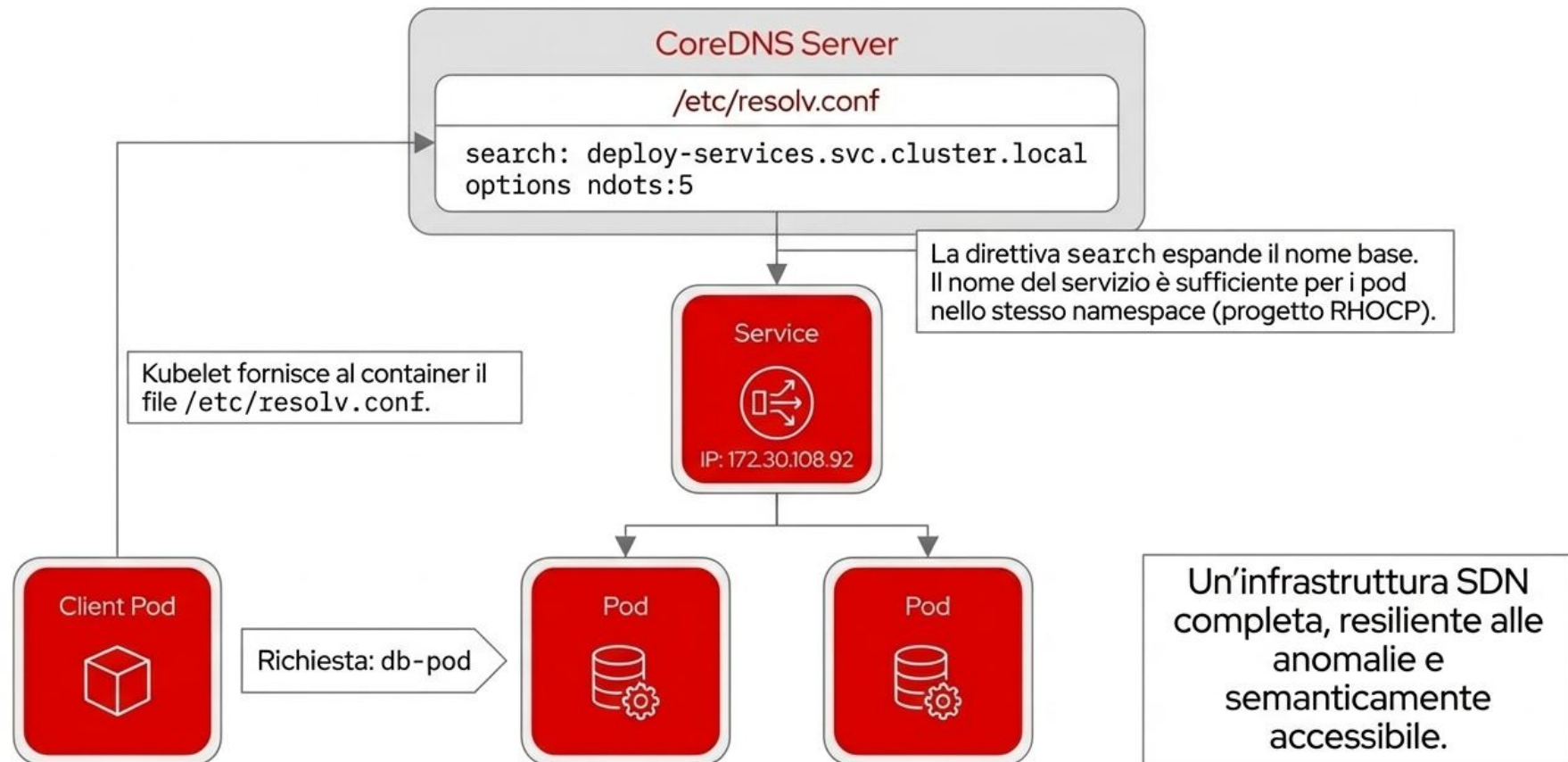


2

Il server DNS interno è visibile esclusivamente ai pod del cluster.



Risoluzione dei Nomi: La Prospettiva del Client



Esposizione del traffico con Load Balancers

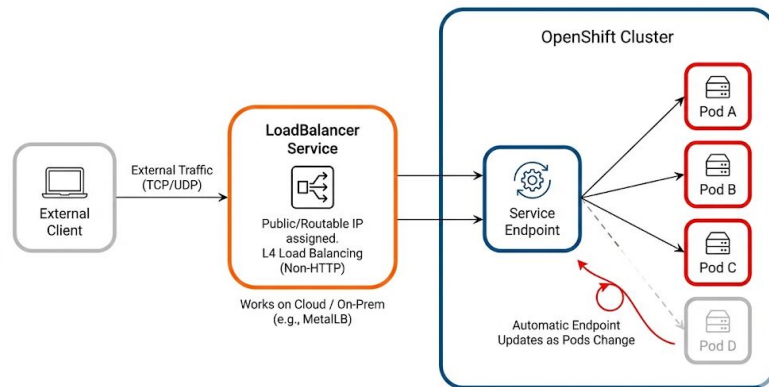
- Espone un Service verso l'esterno assegnandogli un IP pubblico
- Funziona sia su cloud provider sia on-prem tramite MetalLB.
- Bilanciamento L4 (TCP/UDP), ideale per traffico non-HTTP.

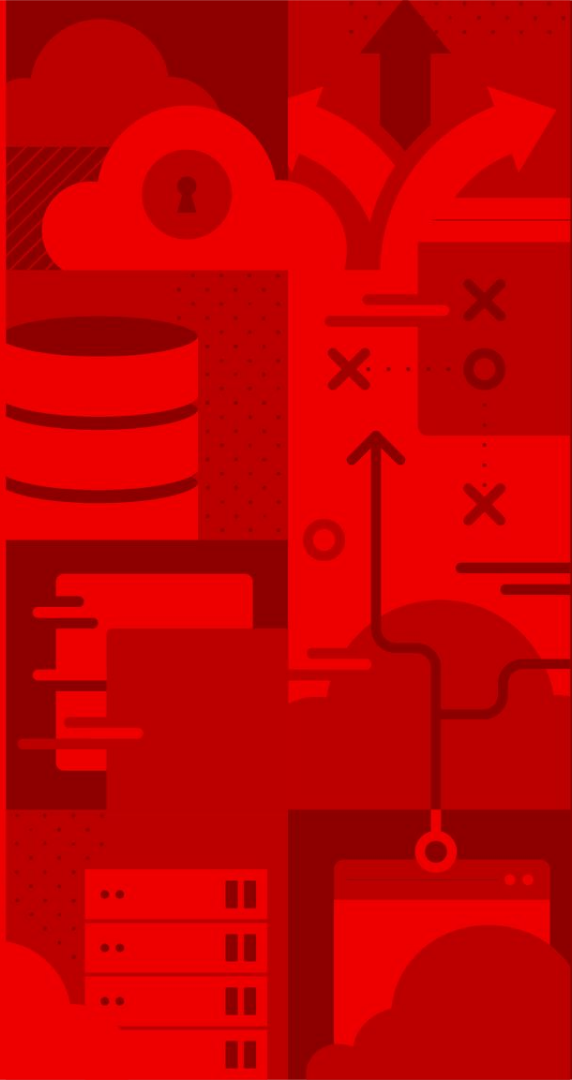
✓ Vantaggi

- Accesso esterno semplice e diretto tramite per ogni protocollo

✗ Svantaggi

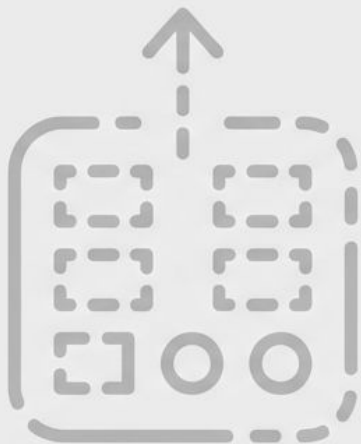
- Richiede un componente esterno che gestisca gli IP (cloud LB o MetalLB).



An abstract graphic on the left side of the slide, composed of various geometric shapes and icons in shades of red and orange. It includes representations of server racks, database cylinders, a cloud with a keyhole, arrows indicating data flow, and a network diagram with nodes and connections.

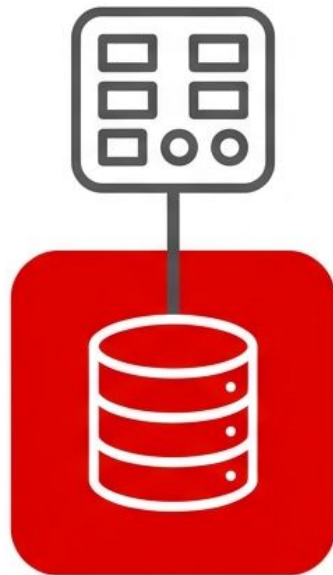
Manage Storage

Il Problema dell'Archiviazione Effimera



Archiviazione Effimera

I container nascono con archiviazione volatile. Se il pod viene distrutto, tutti i dati interni vengono eliminati definitivamente. Nessuna condivisione nativa tra pod differenti.



Storage Persistente

Kubernetes Persistent Storage. Conservazione sicura dei dati indipendente dal ciclo di vita del pod. Astrazione completa dell'infrastruttura sottostante.



Pod (Applicazione)



PersistentVolumeClaim (PVC)



PersistentVolume (PV)



**Infrastruttura di Storage Fisco / Cloud
(NFS, iSCSI, AWS EBS)**

Ruoli e Responsabilità: Amministratori vs. Sviluppatori



Amministratore del Cluster

Gestione dell'Infrastruttura: Configura e mantiene l'hardware fisico o cloud sottostante.

Definizione delle StorageClass: Crea classi per definire i livelli di servizio (QoS, prestazioni, costi).

Provisioning Manuale: Crea preventivamente i PV (Persistent Volumes) se non si utilizza il provisioning dinamico.



Sviluppatore

Astrazione Hardware: Non necessita di competenze specifiche sull'infrastruttura di storage.

Richiesta Risorse (PVC): Crea le PVC (Persistent Volume Claims) per richiedere capacità specifica e modalità di accesso.

Integrazione: Collega semplicemente le PVC ai Pod delle proprie applicazioni tramite i file manifest (YAML).

Metodi di Provisioning: Statico vs. Dinamico

Statico



L'amministratore crea manualmente i PV.

Lo sviluppatore crea la PVC.

Il cluster associa (bind) la PVC al PV preesistente idoneo.

Dinamico



L'amministratore configura le StorageClass.

Lo sviluppatore crea la PVC richiedendo una specifica StorageClass.

Il cluster genera automaticamente il PV su richiesta esatta.

StorageClass: Classificazione e Astrazione dello Storage

Definisce tipi di storage, prestazioni e policy senza esporre i dettagli tecnici agli sviluppatori.



Produzione / Gold (Es. AWS EBS io1, IOPS elevati, massime prestazioni).

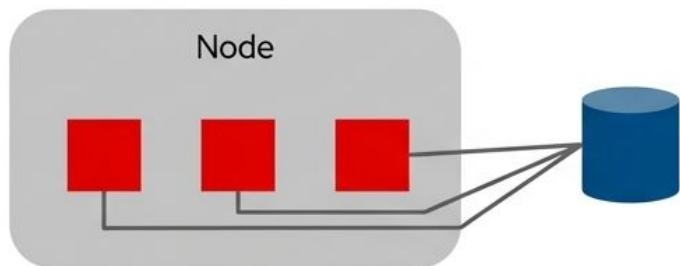


Sviluppo / Standard (Es. Dischi magnetici standard, costo inferiore per carichi di lavoro non critici).

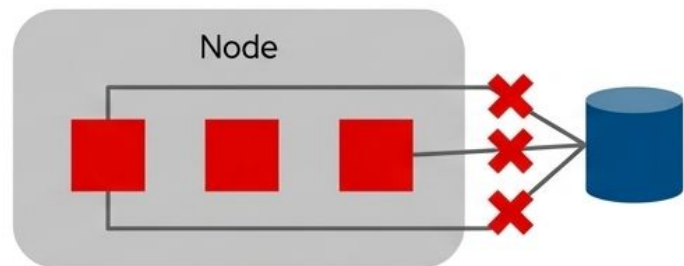


Classe Default (Assegnata automaticamente dal cluster se la PVC dello sviluppatore non specifica alcuna storageClassName).

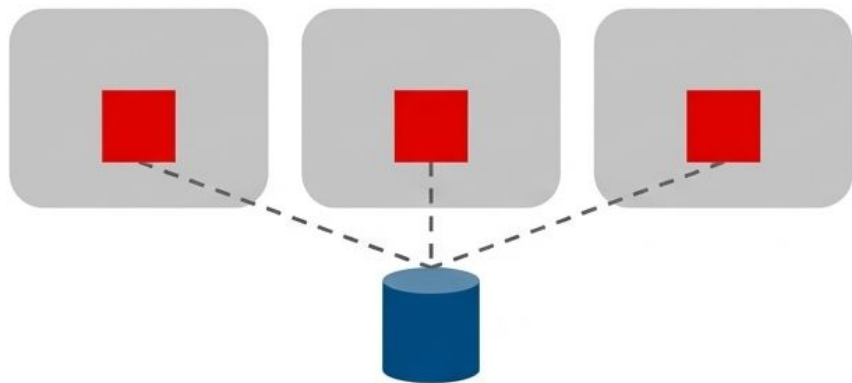
Access Modes: Modalità di Accesso ai Volumi



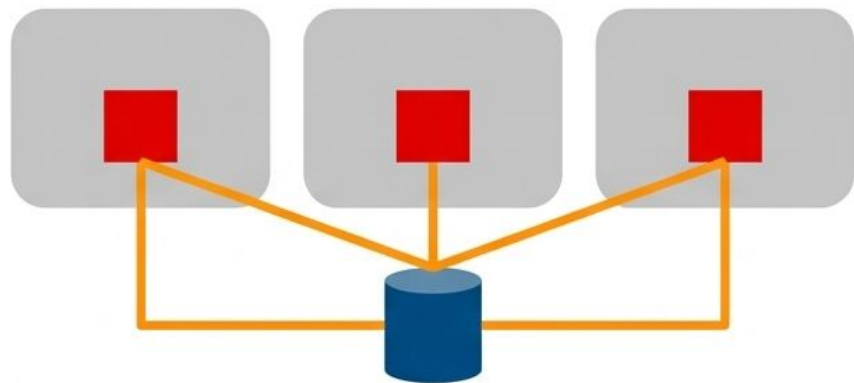
RWO (ReadWriteOnce). 1 Singolo Nodo. Lettura/Scrittura. Condivisibile simultaneamente tra più pod solo se risiedono sullo stesso nodo.



RWOP (ReadWriteOncePod). 1 Singolo Pod. Lettura/Scrittura. Accesso rigorosamente esclusivo.



ROX (ReadOnlyMany). Nodi Multipli. Solo Lettura. Ideale per la distribuzione di configurazioni o dati statici.



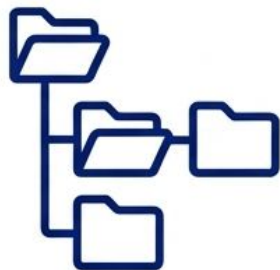
RWX (ReadWriteMany). Nodi Multipli. Lettura/Scrittura concorrente. (Es. supportato da NFS).

Matrice di Supporto Access Modes

Tipo Volume	RWO	RWOP	ROX	RWX
configMap	✓	✓	—	—
emptyDir	✓	✓	—	—
hostPath	✓	✓	—	—
iSCSI	✓	✓	✓	—
local	✓	✓	—	—
NFS	✓	✓	✓	✓

NFS è l'unica opzione standard in questo elenco a supportare l'accesso in scrittura multi-nodo (RWX).

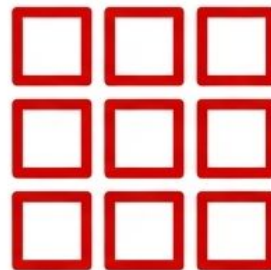
Volume Modes: Filesystem vs. Block



Filesystem (Default)

Il volume viene formattato automaticamente da Kubernetes.

Esposto all'applicazione come una normale directory montata all'interno del container.



Block (Raw)

Nessun filesystem applicato dal cluster (volumeMode: Block).

Il volume è esposto come dispositivo a blocchi grezzo.

Ideale per database ad alte prestazioni che implementano il proprio storage service e scrivono direttamente su disco.

Dinamiche del Ciclo di Vita PV/PVC



Reclaim Policy: Gestione del Rilascio dei Dati

**Azione: L'utente
elimina la PVC**



Retain Policy

Il PV entra in stato 'Released'. I dati fisici vengono conservati. Il volume non è disponibile per nuove claim. L'amministratore deve recuperare lo storage manualmente.



Delete Policy (Default CSI)

Il PV e i dati vengono distrutti definitivamente. L'archiviazione fisica viene rilasciata automaticamente dall'infrastruttura.

Persistent Volume (PV)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
  annotations:
    volume.beta.kubernetes.io/mount-options: rw,nfsvers=4,noexec
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  nfs:
    path: /tmp
    server: 172.17.0.2
  persistentVolumeReclaimPolicy: Retain
  claimRef:
    name: claim1
    namespace: default
```

Specify Mount options (Explained in the next slides)

Set the specific the capacity of the PersistentVolume

Set the access Mode (Explained in the next slides)

Indicates how the resource should be handled once it is released

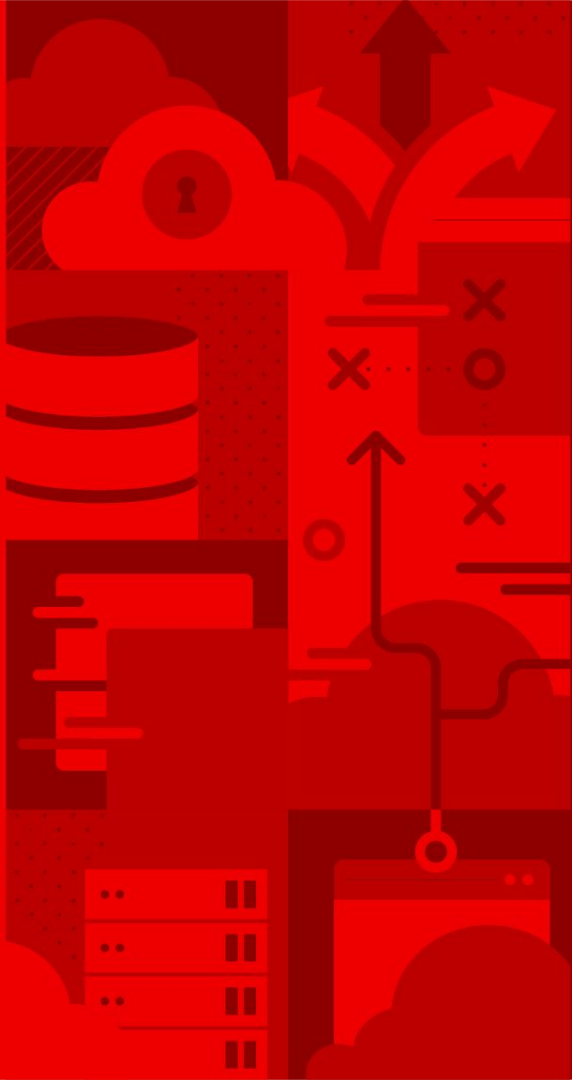
Persistent Volume Claim (PVC)

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  storageClassName: gold
status:
  ...
```

Set the access Mode

Specify the amount of storage

Specify the Storage Class (Explained in the next slides)



High Availability

Chapter 6

Kubernetes High Availability

Kubernetes utilizza le seguenti tecniche per ottenere HA dei workloads:

- **Restart dei Pod:** configurando una restart policy, il cluster riavvia automaticamente le istanze dell'applicazione che si comportano in modo anomalo.
- **Probing:** grazie alle health probe, il cluster rileva quando un'applicazione non risponde e può intervenire automaticamente per mitigare il problema.
- **Horizontal AutoScaler:** quando il carico varia, il cluster può aumentare o ridurre il numero di repliche per adeguarsi alla domanda.

Restart Policy

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  namespace: my-namespace
spec:
  restartPolicy: OnFailure
  containers:
    - name: my-container
      image: httpd:latest
      ports:
        - containerPort: 8080
```

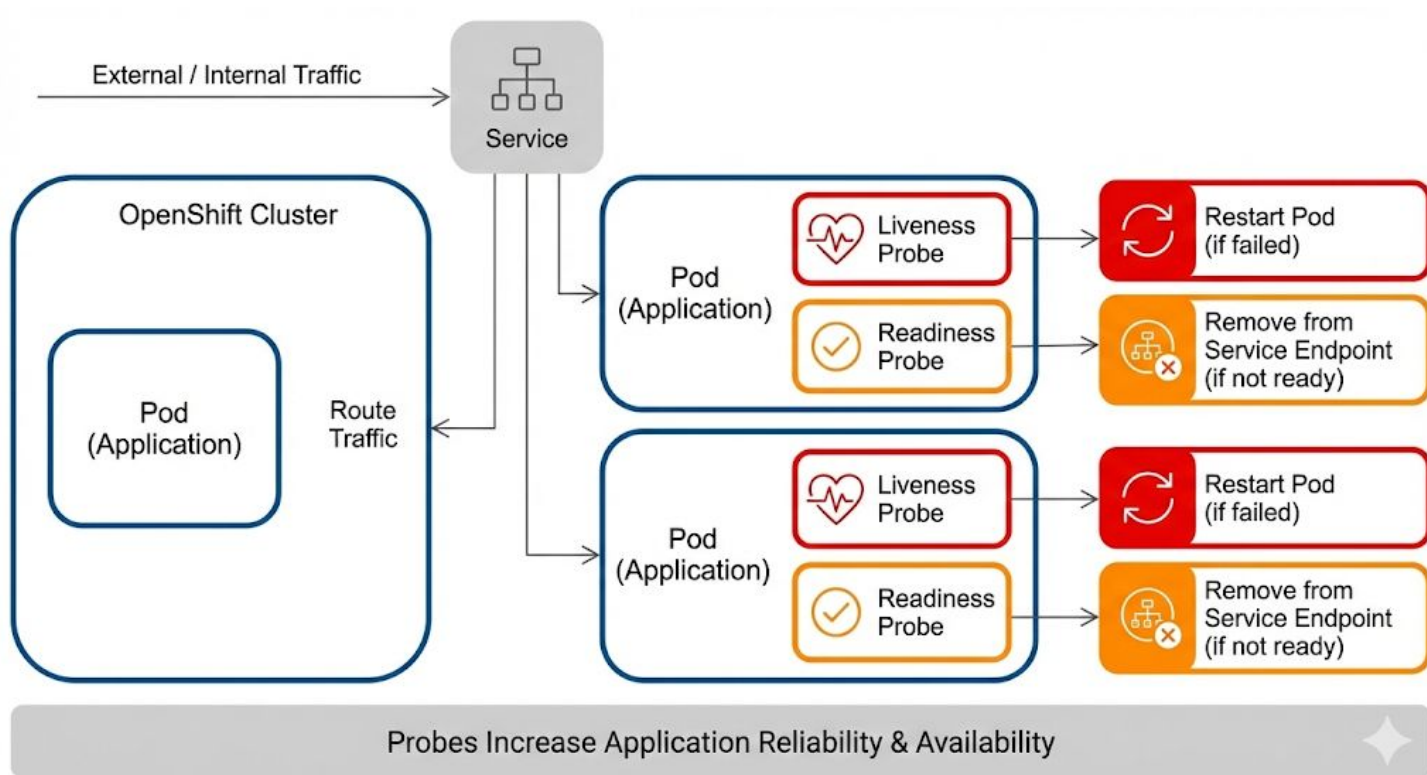
The Pod restarts only if the container exits with a failure (non-zero exit code).



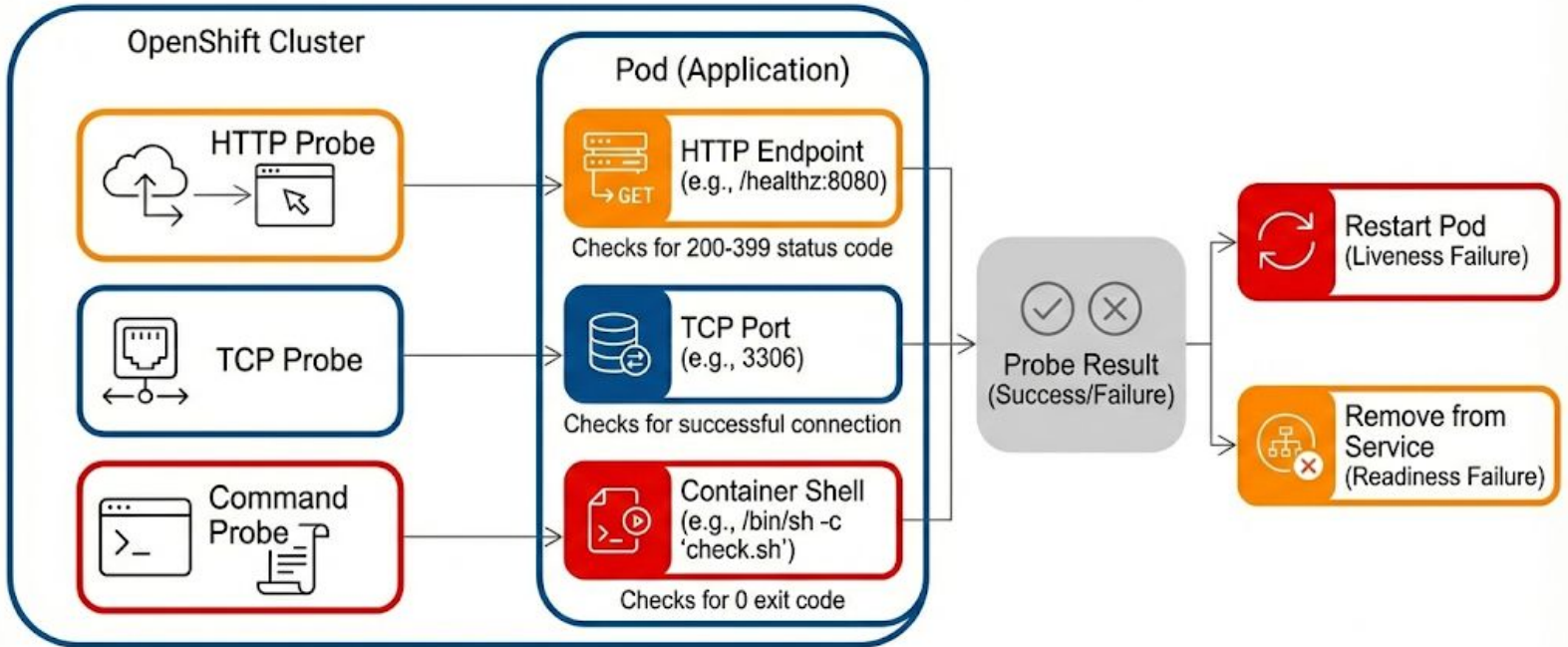
Explanation of restartPolicy Values

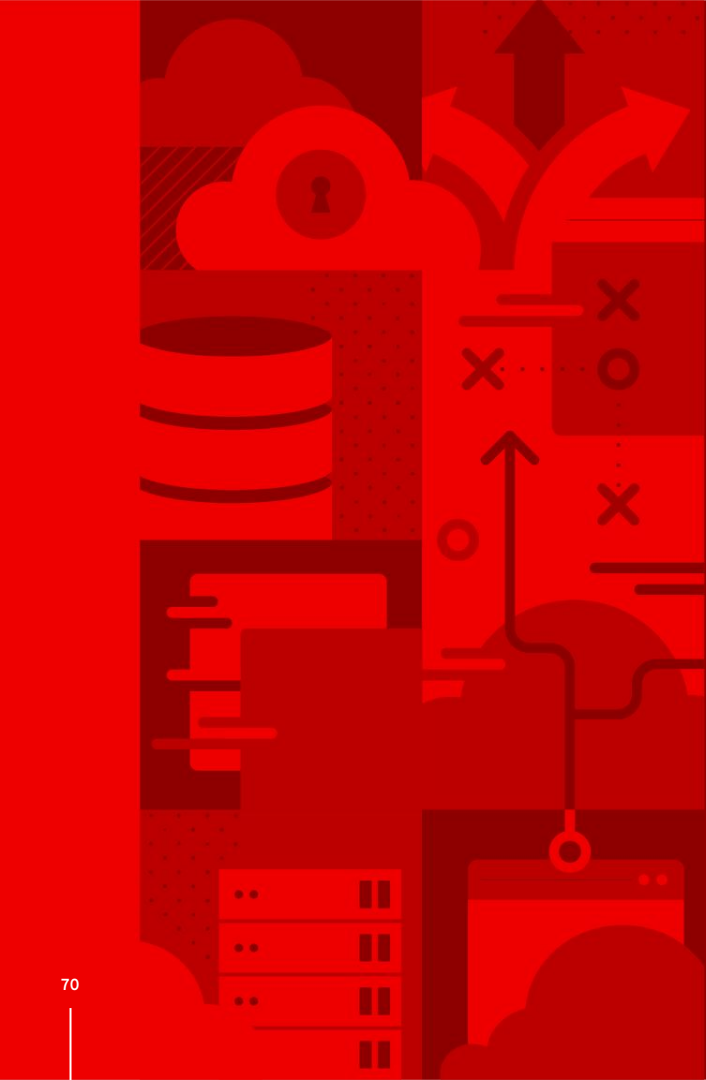
- Always (default) → The Pod restarts automatically if the container stops (even if it exits successfully).
- OnFailure → The Pod restarts only if the container exits with a failure (non-zero exit code).
- Never → The Pod never restarts, even if it fails.

Liveness and Readiness Probes



Probes e Trasporto





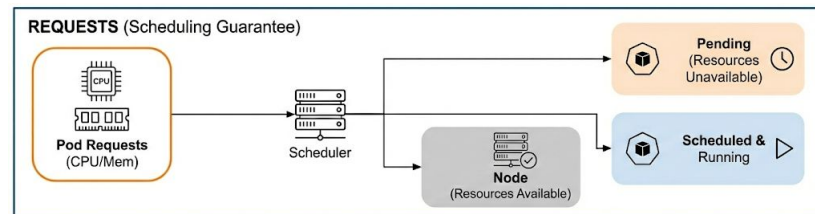
Limit Compute Capacity for Applications

Chapter 6

Gestione risorse dei Workloads

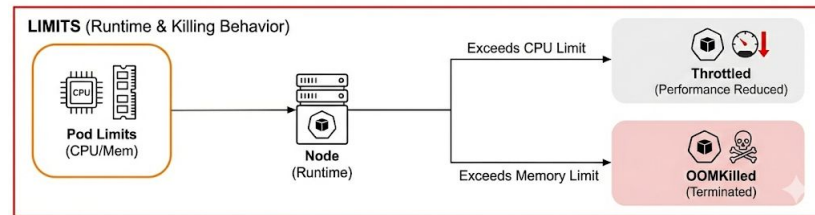
Le *Requests* → influenzano lo Scheduling

- Quantità pianificata di CPU e memoria per un Pod.
- Lo scheduler verifica che il nodo abbia requests sufficienti.
- I Pod restano in Pending se le richieste non possono essere soddisfatte.



I *Limits* → influenzano il Runtime

- Se supera il limite di CPU → il Pod viene throttled (non terminato).
- Se supera il limite di memoria → il Pod viene OOMKilled



Configurazione Requests, Limits

```
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
spec:
  containers:
    - name: app
      image: my-app:latest
      resources:
        requests:
          cpu: "250m"      # Minimum guaranteed CPU (0.25 cores)
          memory: "256Mi" # Minimum guaranteed memory
        limits:
          cpu: "500m"      # Max CPU (can be throttled)
          memory: "512Mi" # Max memory (exceeding kills pod)
```


Resource Quotas

Le Resource Quota sono definite dai cluster admin per stabilire la Quota di risorse per un singolo namespace

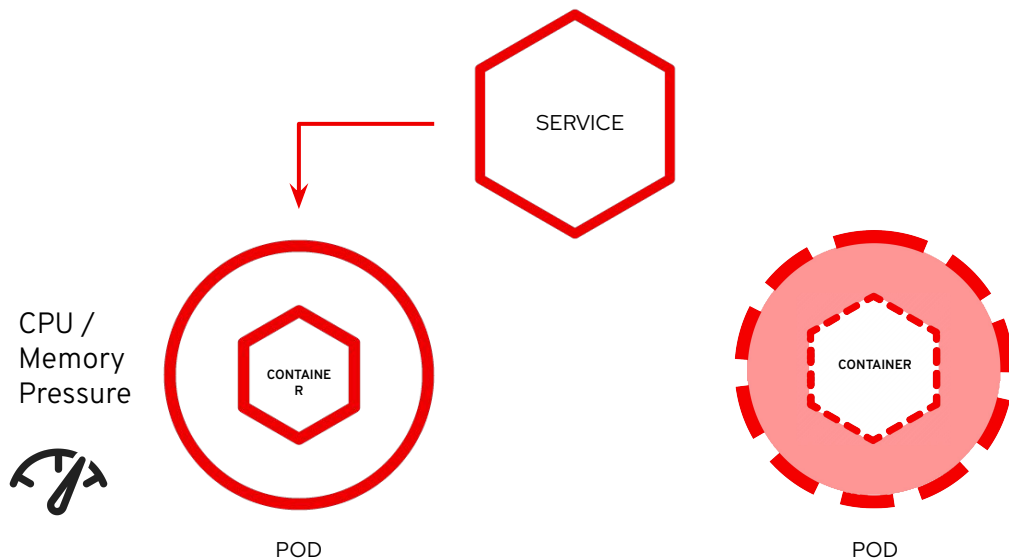
```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-resources
spec:
  hard:
    requests.cpu: "2"
    requests.memory: "4Gi"
    limits.cpu: "4"
    limits.memory: "8Gi"
```



CONTINUES...
IN DO280!

Horizontal Autoscaler

L'Horizontal Pod Autoscaler (HPA) scala automaticamente il numero di pod in un deployment, replica set o stateful set in base a metriche di CPU, memoria o metriche personalizzate.



Come funziona l'HPA

- **Monitora l'utilizzo delle risorse:** l'HPA controlla le metriche (ad esempio CPU e memoria) tramite il Kubernetes Metrics Server.
- **Regola il numero di pod:** se l'utilizzo delle risorse supera la soglia definita, l'HPA aumenta il numero di pod; se l'utilizzo diminuisce, li riduce.
- **Garantisce uno scaling efficiente:** contribuisce a mantenere le prestazioni dell'applicazione ottimizzando al tempo stesso l'uso delle risorse.

```
oc autoscale deployment/hello --min 1 --max 10 --cpu-percent 80
```